

```

01 # ===== FILE: app.py =====
02 from __future__ import annotations
03
04 import os
05 import sys
06 import threading
07 import traceback
08
09 ROOT = os.path.dirname(os.path.abspath(__file__))
10 sys.path.insert(0, ROOT)
11
12 try:
13     from core.paths import app_data_dir, resource_root
14
15     APP_DATA_DIR = app_data_dir()
16     RESOURCE_ROOT = resource_root()
17     os.chdir(APP_DATA_DIR)
18     for path in (str(APP_DATA_DIR), str(RESOURCE_ROOT)):
19         if path not in sys.path:
20             sys.path.insert(0, path)
21 except Exception:
22     APP_DATA_DIR = ROOT
23     RESOURCE_ROOT = ROOT
24
25 from PySide6.QtGui import QIcon
26 from PySide6.QtWidgets import QApplication
27
28 from ui.main_window import MainWindow
29
30 try:
31     from core.driver import cleanup_active_drivers, cleanup_orphan_driver_processes
32 except Exception:
33     cleanup_active_drivers = lambda: None
34     cleanup_orphan_driver_processes = lambda: 0
35
36
37 def _write_crash_log(text: str) -> None:
38     try:
39         log_dir = os.path.join(str(APP_DATA_DIR), "logs")
40         os.makedirs(log_dir, exist_ok=True)
41         with open(os.path.join(log_dir, "crash.log"), "a", encoding="utf-8") as f:
42             f.write(text.rstrip() + "\n\n")
43     except Exception:
44         pass
45
46
47 def _install_crash_cleanup() -> None:
48     old_excepthook = sys.excepthook
49
50     def excepthook(exc_type, exc, tb):

```

```

01         _write_crash_log("".join(traceback.format_exception(exc_type, exc, tb)))
02         cleanup_active_drivers()
03         old_excepthook(exc_type, exc, tb)
04
05     sys.excepthook = excepthook
06
07     if hasattr(threading, "excepthook"):
08         old_threading_hook = threading.excepthook
09
10         def threading_hook(args):
11             _write_crash_log("".join(traceback.format_exception(args.exc_type, args.exc_value, args.exc_traceback)))
12             cleanup_active_drivers()
13             old_threading_hook(args)
14
15         threading.excepthook = threading_hook
16
17
18 def main():
19     cleanup_orphan_driver_processes()
20     _install_crash_cleanup()
21     app = QApplication.instance() or QApplication(sys.argv)
22     app.aboutToQuit.connect(cleanup_active_drivers)
23     app.setStyle("Fusion")
24
25     icon_paths = [
26         os.path.join(str(APP_DATA_DIR), "OIP-C.ico"),
27         os.path.join(str(RESOURCE_ROOT), "OIP-C.ico"),
28     ]
29     app_icon = None
30     for path in icon_paths:
31         if os.path.exists(path):
32             app_icon = QIcon(path)
33             app.setWindowIcon(app_icon)
34             break
35
36     win = MainWindow()
37     if app_icon is not None and not app_icon.isNull():
38         win.setWindowIcon(app_icon)
39     win.show()
40     sys.exit(app.exec())
41
42
43 if __name__ == "__main__":
44     main()
45
46 # ===== FILE: config.py =====
47 # =====
48 # LibSeat Allocator 配置文件
49 # =====
50

```

```

01 # —— 账号 ——
02 USERS = {
03     "": {
04         "password": "",
05     },
06 }
07
08 # —— 目标 ——
09 TARGET_CAMPUS = "崇山校区图书馆"
10 TARGET_ROOM = "三楼智慧研修空间"
11
12 # —— 全天时间段 ——
13 ALLOC_DAY_START = "09:00"
14 ALLOC_DAY_END = "21:00"
15
16 # —— 模式: "multi" = 多账号分时段 | "single" = 单账号逐段 ——
17 ALLOCATION_MODE = "single"
18
19 # —— 测试模式: 只抓取 API 空闲座位并生成方案, 不执行预约 ——
20 ALLOC_DRY_RUN = False
21 API_SCAN_WORKERS = 10
22
23 # —— 多账号模式: 最多账号数 (1-3) ——
24 ALLOC_MAX_ACCOUNTS = 3
25
26 # —— 跨房间搜索 ——
27 ALLOC_CROSS_ROOM = False
28 ALLOC_CROSS_ROOMS = []
29
30 # —— 单账号模式: 提前多少分钟查找下一座位 ——
31 ALLOC_PRE_NOTIFY_MINUTES = 30
32
33 # —— 单账号模式: 自动取消并换座 (无需确认) ——
34 ALLOC_AUTO_CANCEL = False
35
36 # —— 通知超时 (秒) ——
37 ALLOC_NOTIFY_TIMEOUT = 300
38
39 # —— 邮箱通知 ——
40 RECEIVER_EMAIL = ""
41 SMTP_USER = "lnu_library@163.com"
42 SMTP_PASS = "LWQWA366dd2g6msk"
43
44 # —— 浏览器: "edge" | "chrome" ——
45 BROWSER = "edge"
46 DRIVER_PATH = ""
47 WEBDRIVER_CACHE = ""
48
49 # —— 日志 ——
50 LOG_LEVEL = "INFO"

```

```

01 GUI_LOG_LEVEL = "INFO" # GUI 输出过滤级别, 可改为 "DEBUG" 查看更细日志
02 LOG_DIR = "logs"
03
04 # ===== FILE: core/__init__.py =====
05
06 # ===== FILE: core/api_client.py =====
07 """REST API 客户端 — 直接调用 libseat 后端接口, 绕过 Selenium 点击。
08
09 发现的 API 端点 (base: http://libseat.lnu.edu.cn/rest/v2/) :
10 GET /room/stats2/{buildingId}/{date}          — 全校区房间统计 (free/total)
11 GET /room/layoutByDate/{roomId}/{date}       — 房间座位布局 (所有座位ID)
12 GET /startTimesForSeat/{seatId}/{date}       — 可用开始时间
13 GET /endTimesForSeat/{seatId}/{date}/{start} — 可用结束时间
14 POST /room/floors/{buildingId}               — 楼层列表
15
16 认证: URL 参数 ?token=xxx (登录后从页面提取)
17 """
18 import json
19 import hashlib
20 import hmac
21 import requests
22 import threading
23 import uuid
24 from collections import Counter
25 from datetime import datetime, timezone, timedelta
26 from time import perf_counter, time
27 from typing import Any, Dict
28
29 from core.logger import get_logger
30
31 logger = get_logger(__name__)
32
33 BASE_URL = "http://libseat.lnu.edu.cn/rest/v2"
34 _HMAC_SECRET = "leos3cr3t"
35
36
37 def _safe_params(params: dict) -> dict:
38     """隐藏 token/cookie 这类敏感参数, 便于安全打印。"""
39     safe = {}
40     for key, value in (params or {}).items():
41         lowered = str(key).lower()
42         if lowered in ("token", "cookie", "session", "password"):
43             text = str(value)
44             safe[key] = f"{text[:6]}...{text[-4:]}" if len(text) > 12 else "***"
45         else:
46             safe[key] = value
47     return safe
48
49
50 def _compact_json(value: Any, limit: int = 800) -> str:

```

```

01     text = json.dumps(value, ensure_ascii=False, default=str)
02     if len(text) > limit:
03         return text[:limit] + f"... ({len(text)} chars)"
04     return text
05
06
07 def _format_time_option(item: dict) -> str:
08     value = str(item.get("id", ""))
09     if value == "now":
10         return "now"
11     if value.isdigit():
12         mins = int(value)
13         return f"{mins // 60:02d}:{mins % 60:02d}"
14     return value or str(item)[:40]
15
16
17 def _layout_seat_count(layout: dict) -> int:
18     count = 0
19     for pos_data in (layout or {}).values():
20         if not isinstance(pos_data, dict):
21             continue
22         if pos_data.get("type") == "empty":
23             continue
24         if pos_data.get("id") or pos_data.get("seatId"):
25             count += 1
26     return count
27
28
29 def _response_summary(path: str, payload: dict) -> str:
30     if not isinstance(payload, dict):
31         return f"payload={type(payload).__name__}"
32
33     status = payload.get("status", "")
34     message = payload.get("message") or payload.get("msg") or ""
35     data = payload.get("data")
36     parts = []
37     if status:
38         parts.append(f"status={status}")
39     if message:
40         parts.append(f"message={str(message)[:80]}")
41
42     if isinstance(data, list):
43         parts.append(f"data=list[{len(data)}]")
44     elif isinstance(data, dict):
45         parts.append(f"data.keys={','.join(list(data.keys())[:8])}")
46     elif data is not None:
47         parts.append(f"data={type(data).__name__}")
48
49     if "/room/stats2/" in path and isinstance(data, list):
50         samples = []

```

```

01     for room in data[:5]:
02         if not isinstance(room, dict):
03             continue
04         name = room.get("room") or room.get("name") or room.get("roomName") or "?"
05         rid = room.get("roomId") or room.get("id") or "?"
06         free = room.get("free", "?")
07         total = room.get("total", "?")
08         samples.append(f"{name} (id={rid}, free={free}/{total})")
09     if samples:
10         parts.append("rooms=" + "; ".join(samples))
11 elif "/room/layoutByDate/" in path and isinstance(data, dict):
12     layout = data.get("layout", {})
13     parts.append(f"layout_seats={_layout_seat_count(layout)}")
14 elif "/startTimesForSeat/" in path and isinstance(data, dict):
15     starts = data.get("startTimes", [])
16     sample = ", ".join(_format_time_option(item) for item in starts[:6] if isinstance(item, dict))
17     parts.append(f"startTimes={len(starts)}[{sample}]")
18 elif "/endTimesForSeat/" in path and isinstance(data, dict):
19     ends = data.get("endTimes", [])
20     sample = ", ".join(_format_time_option(item) for item in ends[:6] if isinstance(item, dict))
21     parts.append(f"endTimes={len(ends)}[{sample}]")
22
23 return " | ".join(parts) if parts else "no summary"
24
25
26 def _is_seat_time_path(path: str) -> bool:
27     return "/startTimesForSeat/" in path or "/endTimesForSeat/" in path
28
29
30 def _is_empty_seat_time_failure(path: str, payload: dict) -> bool:
31     if not _is_seat_time_path(path) or not isinstance(payload, dict):
32         return False
33     if str(payload.get("status", "")).lower() not in ("fail", "failed"):
34         return False
35     data = payload.get("data")
36     if not isinstance(data, dict):
37         return False
38     if "/startTimesForSeat/" in path:
39         return data.get("startTimes") == []
40     if "/endTimesForSeat/" in path:
41         return data.get("endTimes") == []
42     return False
43
44
45 def _short_error(text: str) -> str:
46     text = (text or "未知原因").strip()
47     return text[:60]
48
49
50 def _sample_available(results: list, limit: int = 5) -> str:

```

```

01  samples = []
02  for item in results[:limit]:
03      starts = ", ".join(item.get("start_times", [])[:2])
04      ends = ", ".join(item.get("end_times", [])[-2:])
05      samples.append(f"{item.get('seat_num')} ({starts}->{ends})")
06  return "; ".join(samples)
07
08
09  def _signed_headers(token: str, method: str) -> dict:
10      request_id = str(uuid.uuid4())
11      request_date = int(time() * 1000)
12      raw = f"seat::{request_id}::{request_date}::{method.upper()}"
13      request_key = hmac.new(_HMAC_SECRET.encode("utf-8"),
14                              raw.encode("utf-8"),
15                              hashlib.sha256).hexdigest()
16      return {
17          "Authorization": token or "",
18          "loginType": "PC",
19          "X-request-id": request_id,
20          "X-request-date": str(request_date),
21          "X-hmac-request-key": request_key,
22      }
23
24
25  def _bj_date() -> str:
26      """返回北京时间日期字符串 YYYY-MM-DD"""
27      now = datetime.now(timezone.utc).astimezone(timezone(timedelta(hours=8)))
28      return now.strftime("%Y-%m-%d")
29
30
31  def _time_to_minutes(t: str) -> int:
32      """HH:MM -> 分钟数"""
33      parts = t.strip().split(":")
34      return int(parts[0]) * 60 + int(parts[1])
35
36
37  def extract_token(driver) -> str:
38      """从已登录的页面提取 API token"""
39      js_code = """
40      // 尝试多种方式提取 token
41      let token = '';
42      function pickStorageToken(storage) {
43          const preferredKeys = [
44              'token', 'libseat-m-token', 'accessToken', 'access_token',
45              'authToken', 'authorization', 'Authorization'
46          ];
47          for (const key of preferredKeys) {
48              try {
49                  const value = storage.getItem(key);
50                  if (value) return value;

```

```

01         } catch(e) {}
02     }
03     try {
04         for (let i = 0; i < storage.length; i++) {
05             const key = storage.key(i) || '';
06             if (key.toLowerCase().includes('token')) {
07                 const value = storage.getItem(key);
08                 if (value) return value;
09             }
10         }
11     } catch(e) {}
12     return '';
13 }
14 // 1. localStorage / sessionStorage (新版网页使用 libseat-m-token)
15 try { token = pickStorageToken(localStorage); } catch(e) {}
16 if (!token) try { token = pickStorageToken(sessionStorage); } catch(e) {}
17 // 2. Vuex store
18 if (!token) try {
19     let app = document.querySelector('#app').__vue__;
20     if (app && app.$store) token = app.$store.state.token || app.$store.state?.user?.token || '';
21 } catch(e) {}
22 // 3. window.__INITIAL_STATE__
23 if (!token) try {
24     token = window.__INITIAL_STATE__?.token || window.__INITIAL_STATE__?.user?.token || '';
25 } catch(e) {}
26 return token;
27 """
28 try:
29     token = driver.execute_script(js_code)
30     if token:
31         logger.info("    Token 已提取: %s...", token[:20])
32         return token
33 except Exception as e:
34     logger.warning("    Token 提取失败: %s", e)
35
36 # 兜底: 从页面请求中抓取 (网络嗅探器)
37 try:
38     collector = getattr(driver, "__sniffer_collect", None)
39     if collector:
40         requests_list = collector()
41         for r in requests_list:
42             url = r.get("url", "")
43             if "token=" in url:
44                 # 从 URL 中提取 token 参数
45                 import re
46                 match = re.search(r'token=([a-f0-9]+)', url)
47                 if match:
48                     token = match.group(1)
49                     logger.info("    Token 从网络请求中提取: %s...", token[:20])
50                     return token

```



```

01     except Exception:
02         pass
03
04     logger.error("    无法提取 token")
05     return ""
06
07
08 class APIClient:
09     """libseat REST API 客户端"""
10
11     def __init__(self, token: str, driver=None, base_url: str = BASE_URL):
12         self.token = token
13         self.driver = driver
14         self._browser_lock = threading.Lock()
15         self.base_url = base_url
16         self.session = requests.Session()
17         self.session.trust_env = False # 避免系统代理干扰校内 HTTP 接口
18         self.session.headers.update({
19             "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.3 ...
20             "Referer": "http://libseat.lnu.edu.cn/",
21             "Accept": "application/json, text/plain, */*",
22             "Accept-Language": "zh-CN,zh;q=0.9",
23             "Connection": "keep-alive",
24         })
25         # 从 Selenium 浏览器复制 Cookies
26         if driver:
27             self._copy_cookies(driver)
28
29     def _copy_cookies(self, driver):
30         """将 Selenium 浏览器的 Cookies 复制到 requests Session"""
31         try:
32             selenium_cookies = driver.get_cookies()
33             for cookie in selenium_cookies:
34                 self.session.cookies.set(
35                     cookie.get("name", ""),
36                     cookie.get("value", ""),
37                     domain=cookie.get("domain", ""),
38                     path=cookie.get("path", "/"),
39                 )
40             logger.info("? 已复制 %d 个 Cookies", len(selenium_cookies))
41         except Exception as e:
42             logger.warning("    复制 Cookies 失败: %s", e)
43
44     def _get(self, path: str, params: dict = None) -> dict:
45         """发送 GET 请求"""
46         params = dict(params or {})
47         params["token"] = self.token
48         return self._request("GET", path, params=params)
49
50     def _post(self, path: str, data: dict = None) -> dict:

```

```
01     """发送 POST 请求"""
02     params = {"token": self.token}
03     return self._request("POST", path, params=params, data=data or {})
04
05 def _request(self, method: str, path: str, params: dict = None, data: dict = None) -> dict:
06     """发送请求并输出过滤后的 API 摘要。"""
07     url = f"{self.base_url}{path}"
08     params = dict(params or {})
09     method = method.upper()
10     detail_log = logger.debug if _is_seat_time_path(path) else logger.info
11     detail_log("? [API] %s %s params=%s", method, path, _safe_params(params))
12     headers = _signed_headers(self.token, method)
13
14     started = perf_counter()
15     try:
16         request_timeout = 6 if _is_seat_time_path(path) else 10
17         if method == "GET":
18             resp = self.session.get(url, params=params, headers=headers, timeout=request_timeout)
19         else:
20             resp = self.session.post(url, params=params, json=data or {}, headers=headers, timeout=request_timeout)
21
22         elapsed_ms = int((perf_counter() - started) * 1000)
23         try:
24             payload = resp.json()
25         except ValueError:
26             preview = (resp.text or "").replace("\n", " ")[:300]
27             logger.warning(
28                 "[API] %s %s -> HTTP %d %dms | 非 JSON 响应: %s",
29                 method, path, resp.status_code, elapsed_ms, preview,
30             )
31             return {"status": "error", "message": f"非 JSON 响应: {preview}"}
32
33         summary = _response_summary(path, payload)
34         payload_status = str(payload.get("status", "")).lower() if isinstance(payload, dict) else ""
35         if not resp.ok or payload_status in ("error", "fail", "failed"):
36             message = payload.get("message") if isinstance(payload, dict) else resp.reason
37             if self._needs_browser_fallback(payload):
38                 browser_payload = self._browser_request(method, path, params=params, data=data)
39                 if browser_payload.get("status") != "error":
40                     return browser_payload
41             log_result = logger.debug if resp.ok and _is_empty_seat_time_failure(path, payload) else logger.warning
42             log_result(
43                 "[API] %s %s -> HTTP %d %dms | %s",
44                 method, path, resp.status_code, elapsed_ms, summary,
45             )
46             return {"status": "error", "message": str(message or resp.reason), "data": payload}
47
48     detail_log(
49         "[API] %s %s -> HTTP %d %dms | %s",
50         method, path, resp.status_code, elapsed_ms, summary,
```

```

01         )
02         logger.debug("? [API] %s %s body=%s", method, path, _compact_json(payload))
03         return payload
04     except requests.Timeout as e:
05         logger.warning("    [API] %s %s 请求超时: %s", method, path, e)
06         return {"status": "error", "message": str(e), "timeout": True}
07     except requests.RequestException as e:
08         logger.warning("    [API] %s %s 请求失败: %s", method, path, e)
09         return {"status": "error", "message": str(e)}
10     except Exception as e:
11         logger.warning("    [API] %s %s 异常: %s", method, path, e)
12         return {"status": "error", "message": str(e)}
13
14     def _needs_browser_fallback(self, payload: dict) -> bool:
15         if not self.driver or not isinstance(payload, dict):
16             return False
17         message = str(payload.get("message") or "")
18         return payload.get("code") == "20" or "app已停用" in message or "小程序" in message
19
20     def _browser_request(self, method: str, path: str, params: dict = None, data: dict = None) -> dict:
21         """在已登录浏览器上下文中调用同源 API，绕过服务端对非浏览器请求的拒绝。"""
22         url = f"{self.base_url}{path}"
23         params = dict(params or {})
24         method = method.upper()
25         detail_log = logger.debug if _is_seat_time_path(path) else logger.info
26         detail_log("? [API/browser] %s %s params=%s", method, path, _safe_params(params))
27         signed_headers = _signed_headers(self.token, method)
28
29         js = """
30         const done = arguments[arguments.length - 1];
31         const method = arguments[0];
32         const url = arguments[1];
33         const params = arguments[2] || {};
34         const data = arguments[3] || {};
35         const signedHeaders = arguments[4] || {};
36         const query = new URLSearchParams(params).toString();
37         const target = url + (query ? (url.includes('?') ? '&' : '?') + query : '');
38         const options = {
39             method,
40             credentials: 'include',
41             headers: Object.assign({
42                 'Accept': 'application/json, text/plain, */*',
43                 'Content-Type': 'application/json;charset=UTF-8'
44             }, signedHeaders)
45         };
46         if (method !== 'GET') {
47             options.body = JSON.stringify(data || {});
48         }
49         fetch(target, options)
50             .then(async (resp) => done({

```

```

01         ok: resp.ok,
02         status: resp.status,
03         text: await resp.text()
04     )))
05     .catch((err) => done({
06         ok: false,
07         status: 0,
08         error: String(err)
09     }));
10     """
11
12     with self._browser_lock:
13         try:
14             self.driver.set_script_timeout(15)
15         except Exception:
16             pass
17         try:
18             result = self.driver.execute_async_script(js, method, url, params, data or {}, signed_headers)
19         except Exception as exc:
20             logger.warning("    [API/browser] %s %s 调用失败: %s", method, path, exc)
21             return {"status": "error", "message": str(exc)}
22
23     if result.get("error"):
24         logger.warning("    [API/browser] %s %s fetch失败: %s", method, path, result["error"])
25         return {"status": "error", "message": result["error"]}
26
27     try:
28         payload = json.loads(result.get("text") or "{}")
29     except ValueError:
30         preview = (result.get("text") or "").replace("\n", " ")[:300]
31         logger.warning(
32             "    [API/browser] %s %s -> HTTP %s | 非 JSON 响应: %s",
33             method, path, result.get("status"), preview,
34         )
35         return {"status": "error", "message": f"非 JSON 响应: {preview}"}
36
37     summary = _response_summary(path, payload)
38     payload_status = str(payload.get("status", "")).lower() if isinstance(payload, dict) else ""
39     if not result.get("ok") or payload_status in ("error", "fail", "failed"):
40         logger.warning(
41             "    [API/browser] %s %s -> HTTP %s | %s",
42             method, path, result.get("status"), summary,
43         )
44         return {"status": "error", "message": str(payload.get("message", "")), "data": payload}
45
46     detail_log(
47         "    [API/browser] %s %s -> HTTP %s | %s",
48         method, path, result.get("status"), summary,
49     )
50     logger.debug("? [API/browser] %s %s body=%s", method, path, _compact_json(payload))

```

```
01         return payload
02
03     # ----- 高层 API -----
04
05     def get_floors(self, building_id: int) -> list:
06         """获取校区的楼层列表"""
07         resp = self._post(f"/room/floors/{building_id}")
08         return resp.get("data", [])
09
10     def get_room_stats(self, building_id: int, date: str = "") -> list:
11         """获取校区所有房间的统计信息（含空闲座位数 free）"""
12         if not date:
13             date = _bj_date()
14         resp = self._get(f"/room/stats2/{building_id}/{date}",
15                          params={"buildingId": building_id, "date": date})
16         return resp.get("data", [])
17
18     def get_room_layout(self, room_id: int, date: str = "") -> dict:
19         """获取房间座位布局，含所有座位ID"""
20         if not date:
21             date = _bj_date()
22         resp = self._get(f"/room/layoutByDate/{room_id}/{date}",
23                          params={"id": room_id, "date": date})
24         return resp.get("data", {})
25
26     def get_start_times(self, seat_id: int, date: str = "") -> dict:
27         """获取座位的可用开始时间"""
28         if not date:
29             date = _bj_date()
30         resp = self._get(f"/startTimesForSeat/{seat_id}/{date}",
31                          params={"seatId": seat_id, "date": date})
32         return resp
33
34     def get_end_times(self, seat_id: int, date: str = "", start: str = "") -> dict:
35         """获取座位的可用结束时间（基于选定的开始时间）"""
36         if not date:
37             date = _bj_date()
38         resp = self._get(f"/endTimesForSeat/{seat_id}/{date}/{start}",
39                          params={"id": seat_id, "date": date, "start": start})
40         return resp
41
42     def get_seat_availability(self, seat_id: int, date: str = "") -> dict:
43         """一站式获取单个座位的完整可用时间信息"""
44         result = {"seat_id": str(seat_id), "available": False,
45                  "start_times": [], "end_times": [], "error": ""}
46
47         # 获取开始时间
48         start_resp = self.get_start_times(seat_id, date)
49         if start_resp.get("status") != "success":
50             result["error"] = f"获取开始时间失败: {start_resp.get('message', '')}"
```

```
01         return result
02
03     start_data = start_resp.get("data", {})
04     starts_raw = start_data.get("startTimes", [])
05
06     if not starts_raw:
07         result["error"] = "无可用开始时间"
08         return result
09
10     # 解析开始时间
11     starts = []
12     for s in starts_raw:
13         sid = s.get("id", "")
14         if sid == "now":
15             # "现在" → 当前北京时间
16             now = datetime.now(timezone.utc).astimezone(timezone(timedelta(hours=8)))
17             starts.append(now.strftime("%H:%M"))
18         elif sid.isdigit():
19             mins = int(sid)
20             starts.append(f"{mins // 60:02d}:{mins % 60:02d}")
21
22     if not starts:
23         result["error"] = "开始时间为空"
24         return result
25
26     result["start_times"] = starts
27
28     # 获取结束时间（基于最早可用开始时间）
29     best_start = starts[0]
30     start_param = start_data.get("startTimes", [{}])[0].get("id", best_start)
31     end_resp = self.get_end_times(seat_id, date, start_param)
32     if end_resp.get("status") != "success":
33         result["error"] = f"获取结束时间失败: {end_resp.get('message', '')}"
34         return result
35
36     ends_data = end_resp.get("data", {}).get("endTimes", [])
37     ends = []
38     for e in ends_data:
39         eid = str(e.get("id", ""))
40         if eid.isdigit():
41             mins = int(eid)
42             ends.append(f"{mins // 60:02d}:{mins % 60:02d}")
43
44     if not ends:
45         result["error"] = "结束时间为空"
46         return result
47
48     result["end_times"] = ends
49     result["available"] = True
50     return result
```

```

01
02 def scan_room_api(self, room_id: int, room_name: str, date: str = "",
03     prefer_seats: list = None) -> list:
04     """通过 API 扫描整个房间的所有座位可用时间。
05
06     流程:
07         1. 获取房间布局 → 提取所有座位ID
08         2. 并发获取每个座位的开始/结束时间
09         3. 过滤并排序结果
10
11     Args:
12         room_id: 房间ID (从 room/stats2 获取)
13         room_name: 房间名称
14         date: 日期 YYYY-MM-DD
15         prefer_seats: 优先座位号列表 (可选)
16
17     Returns:
18         [{"seat_num": "92", "room_name": "...", "available": True,
19            "start_times": [...], "end_times": [...]}, ...]
20     """
21     import concurrent.futures
22
23     logger.info("? [API] 扫描房间 %s (ID=%d)...", room_name, room_id)
24
25     # 1. 获取布局
26     layout_data = self.get_room_layout(room_id, date)
27     if not layout_data:
28         logger.error(" [API] 无法获取房间 %s 布局", room_name)
29         return []
30
31     layout = layout_data.get("layout", {})
32     # 从 layout 中提取所有座位ID (type != "empty" 的条目)
33     seat_labels = {}
34     for pos_key, pos_data in layout.items():
35         if not isinstance(pos_data, dict):
36             continue
37         if pos_data.get("type") == "empty":
38             continue
39         sid = pos_data.get("id") or pos_data.get("seatId")
40         if sid:
41             seat_id = int(sid)
42             seat_label = (
43                 pos_data.get("name")
44                 or pos_data.get("seatName")
45                 or pos_data.get("label")
46                 or str(seat_id)
47             )
48             seat_labels[seat_id] = str(seat_label)
49
50     seat_ids = set(seat_labels)

```

```
01
02     if not seat_ids:
03         logger.warning("[API] 房间 %s 布局中未找到座位", room_name)
04         return []
05
06     logger.info("[API] 房间 %s 共 %d 个座位, 开始API扫描...", room_name, len(seat_ids))
07
08     # 2. 优先座位先扫
09     prefer_set = set()
10     if prefer_seats:
11         prefer_set = {int(s) for s in prefer_seats if str(s).isdigit()}
12
13     preferred_ids = [s for s in seat_ids if s in prefer_set]
14     normal_ids = [s for s in seat_ids if s not in prefer_set]
15
16     results = []
17     errors = Counter()
18
19     def collect_result(info):
20         if not info:
21             errors["无返回"] += 1
22             return
23         if info.get("available"):
24             seat_id = int(info.get("seat_id") or info.get("seat_num"))
25             info["seat_id"] = str(seat_id)
26             info["seat_num"] = seat_labels.get(seat_id, str(seat_id))
27             info["room_name"] = room_name
28             results.append(info)
29         else:
30             errors[_short_error(info.get("error"))] += 1
31
32     def scan_one(seat_id):
33         try:
34             info = self.get_seat_availability(seat_id, date)
35             info["seat_id"] = str(seat_id)
36             return info
37         except Exception as e:
38             logger.debug("[API] 座位 %d 扫描异常: %s", seat_id, e)
39             return {"seat_id": str(seat_id), "available": False,
40                     "error": f"扫描异常: {type(e).__name__}"}
41
42     # 优先座位先扫描
43     for sid in preferred_ids:
44         r = scan_one(sid)
45         collect_result(r)
46         if r and r.get("available"):
47             logger.info("★ [API] 优先座位 %d 可用: %s", sid, r["end_times"])
48
49     # 再扫其余座位
50     with concurrent.futures.ThreadPoolExecutor(max_workers=10) as executor:
```



```

01         futures = {executor.submit(scan_one, sid): sid for sid in normal_ids}
02         for future in concurrent.futures.as_completed(futures):
03             collect_result(future.result())
04
05         logger.info("    [API] %s 扫描完成: %d/%d 可用",
06                     room_name, len(results), len(seat_ids))
07     if results:
08         logger.info("? [API] %s 可用样例: %s", room_name, _sample_available(results))
09     if errors:
10         error_summary = "; ".join(f"{reason}={count}" for reason, count in errors.most_common(5))
11         logger.info("    [API] %s 不可用/失败原因: %s", room_name, error_summary)
12     return results
13
14
15 # ----- 全局便捷函数 -----
16
17 _client_cache: Dict[str, APIClient] = {}
18
19
20 def get_client(token: str, driver=None) -> APIClient:
21     """获取或创建 API 客户端（缓存）。driver 用于复制 Cookies。"""
22     if token not in _client_cache:
23         _client_cache[token] = APIClient(token, driver=driver)
24     elif driver:
25         _client_cache[token].driver = driver
26         if not _client_cache[token].session.cookies:
27             _client_cache[token]._copy_cookies(driver)
28     return _client_cache[token]
29
30 # ===== FILE: core/captcha.py =====
31 import dddocr
32 from PIL import Image
33 from io import BytesIO
34 import base64
35 from core.logger import get_logger
36
37 logger = get_logger(__name__)
38
39
40 class CaptchaSolver:
41     def __init__(self):
42         # 初始化一次，避免重复加载模型
43         self.ocr = dddocr.DdddOcr(det=False, use_gpu=False, show_ad=False)
44
45     def solve_base64(self, base64_str):
46         """解析 Base64 图片流"""
47         try:
48             image_bytes = base64.b64decode(base64_str)
49             image = Image.open(BytesIO(image_bytes))
50             return self.ocr.classification(image)

```

```

01         except Exception as e:
02             logger.error("    验证码解析失败: %s", e)
03             return ""
04
05
06 # 单例模式，全局用这就行
07 solver = CaptchaSolver()
08
09 # ===== FILE: core/captcha_click1_yolo4_siamese.py =====
10 """
11 Click1 YOLO4 + Siamese 验证码求解器。
12
13 1 个 plan 裁剪 × 4 个 YOLO 候选 → Siamese argmax → 1 个坐标。
14 端到端准确率 83.48%，YOLO 覆盖率 100%。
15
16 用法:
17     from core.captcha_click1_yolo4_siamese import solve_click1_target_bg, preload_click1_target_bg
18 """
19 from __future__ import annotations
20
21 from dataclasses import dataclass
22 from io import BytesIO
23 from pathlib import Path
24 from typing import Any
25
26 import cv2
27 import numpy as np
28 import onnxruntime as ort
29 from PIL import Image
30
31 from core.yolo_onnx import YoloOnnxPredictor
32
33
34 @dataclass(frozen=True)
35 class Click1Detection:
36     conf: float
37     x1: float
38     y1: float
39     x2: float
40     y2: float
41
42     @property
43     def center(self) -> tuple[float, float]:
44         return (self.x1 + self.x2) / 2.0, (self.y1 + self.y2) / 2.0
45
46
47 @dataclass(frozen=True)
48 class Click1SiameseResult:
49     point: tuple[int, int]
50     candidates: list[tuple[int, int]]

```

```
01     candidate_confidences: list[float]
02     similarity: list[float]
03     best_index: int
04
05
06 def _project_root() -> Path:
07     try:
08         from core.paths import resource_root
09         return resource_root()
10     except Exception:
11         return Path(__file__).resolve().parents[1]
12
13
14 def _resolve_path(path: str | Path) -> Path:
15     p = Path(path)
16     return p if p.is_absolute() else _project_root() / p
17
18
19 def _sigmoid(x: np.ndarray) -> np.ndarray:
20     return 1.0 / (1.0 + np.exp(-x))
21
22
23 def _crop_with_padding(
24     image: np.ndarray,
25     x1: int,
26     y1: int,
27     x2: int,
28     y2: int,
29     fill: tuple[int, int, int],
30 ) -> np.ndarray:
31     h, w = image.shape[:2]
32     canvas = np.full((max(1, y2 - y1), max(1, x2 - x1), 3), fill, dtype=np.uint8)
33     sx1, sy1 = max(0, x1), max(0, y1)
34     sx2, sy2 = min(w, x2), min(h, y2)
35     if sx2 > sx1 and sy2 > sy1:
36         canvas[sy1 - y1 : sy2 - y1, sx1 - x1 : sx2 - x1] = image[sy1:sy2, sx1:sx2]
37     return canvas
38
39
40 def _preprocess_siamese(image_bgr: np.ndarray, input_size: tuple[int, int] = (112, 112)) -> np.ndarray:
41     rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
42     h, w = input_size
43     ih, iw = rgb.shape[:2]
44     scale = min(w / iw, h / ih)
45     nw, nh = int(iw * scale), int(ih * scale)
46     resized = cv2.resize(rgb, (nw, nh), interpolation=cv2.INTER_CUBIC)
47     canvas = np.full((h, w, 3), 128, dtype=np.uint8)
48     dx, dy = (w - nw) // 2, (h - nh) // 2
49     canvas[dy : dy + nh, dx : dx + nw] = resized
50     arr = canvas.astype(np.float32) / 255.0
```

```

01     return np.transpose(arr, (2, 0, 1))
02
03
04 class ClicklYolo4SiameseSolver:
05     """End-to-end clickl solver: target+bg -> YOLO four candidates -> Siamese argmax -> 1 point."""
06
07     def __init__(
08         self,
09         yolo_path: str | Path | None = None,
10         siamese_path: str | Path | None = None,
11         yolo_imgsz: int = 640,
12         yolo_conf: float = 0.05,
13         yolo_iou: float = 0.7,
14         yolo_device: str = "cpu",
15         siamese_device: str = "cpu",
16         top_k: int = 4,
17         char_crop_size: int = 60,
18         plan_left: int = 25,
19         plan_right: int = 45,
20         plan_top: int = 5,
21         plan_bottom: int = 27,
22         plan_slot_height: int = 44,
23     ) -> None:
24         if yolo_path is None:
25             yolo_path = "core/checkpoints/clickl_yolo_plan_bg_char40_best.onnx"
26         if siamese_path is None:
27             siamese_path = "core/checkpoints/clickl_siamese_yolo4_best.onnx"
28
29         self.yolo_path = _resolve_path(yolo_path)
30         self.siamese_path = _resolve_path(siamese_path)
31         self.yolo_imgsz = int(yolo_imgsz)
32         self.yolo_conf = float(yolo_conf)
33         self.yolo_iou = float(yolo_iou)
34         self.top_k = int(top_k)
35         self.char_crop_size = int(char_crop_size)
36         self.plan_left = int(plan_left)
37         self.plan_right = int(plan_right)
38         self.plan_top = int(plan_top)
39         self.plan_bottom = int(plan_bottom)
40         self.plan_slot_height = int(plan_slot_height)
41
42         self._yolo: YoloOnnxPredictor | None = None
43         self._siamese_ort: ort.InferenceSession | None = None
44
45     def _ensure_yolo(self) -> YoloOnnxPredictor:
46         if self._yolo is None:
47             self._yolo = YoloOnnxPredictor(
48                 self.yolo_path, conf=self.yolo_conf, iou=self.yolo_iou, imgsz=self.yolo_imgsz,
49             )
50         return self._yolo

```

```
01
02 def _ensure_siamese_ort(self) -> ort.InferenceSession:
03     if self._siamese_ort is None:
04         if not self.siamese_path.exists():
05             raise FileNotFoundError(f"Siamese ONNX not found: {self.siamese_path}")
06         if self.siamese_path.suffix.lower() != ".onnx":
07             raise ValueError(f"Production Siamese model must be ONNX: {self.siamese_path}")
08         self._siamese_ort = ort.InferenceSession(str(self.siamese_path), providers=["CPUExecutionProvider"])
09     return self._siamese_ort
10
11 def warmup(self) -> None:
12     self._ensure_yolo()
13     self._ensure_siamese_ort()
14     plan_crop = np.full(
15         (max(1, self.plan_bottom - self.plan_top), max(1, self.plan_right - self.plan_left), 3),
16         255,
17         dtype=np.uint8,
18     )
19     bg_bgr = np.full((120, 300, 3), 255, dtype=np.uint8)
20     composed = self._compose_plan_bg(plan_crop, bg_bgr)
21     self._ensure_yolo().predict(composed, max_det=1)
22     char_crops = [
23         np.full((self.char_crop_size, self.char_crop_size, 3), 128, dtype=np.uint8)
24         for _ in range(4)
25     ]
26     self._score_siamese(plan_crop, char_crops)
27
28 def _extract_plan_crop(self, target_bgr: np.ndarray) -> np.ndarray:
29     return _crop_with_padding(
30         target_bgr,
31         self.plan_left,
32         self.plan_top,
33         self.plan_right,
34         self.plan_bottom,
35         fill=(255, 255, 255),
36     )
37
38 def _compose_plan_bg(self, plan_crop: np.ndarray, bg_bgr: np.ndarray) -> np.ndarray:
39     bg_h, bg_w = bg_bgr.shape[:2]
40     canvas = np.full((self.plan_slot_height + bg_h, bg_w, 3), 255, dtype=np.uint8)
41     ch, cw = plan_crop.shape[:2]
42     x = (bg_w - cw) // 2
43     y = (self.plan_slot_height - ch) // 2
44     canvas[y : y + ch, x : x + cw] = plan_crop
45     canvas[self.plan_slot_height : self.plan_slot_height + bg_h, 0:bg_w] = bg_bgr
46     return canvas
47
48 def _detect_candidates(self, composed_bgr: np.ndarray) -> list[Click1Detection]:
49     model = self._ensure_yolo()
50     raw = model.predict(composed_bgr, max_det=300)
```

```

01     detections: list[ClickDetection] = []
02     image_h, image_w = composed_bgr.shape[:2]
03     for det in raw:
04         cx = (det.x1 + det.x2) / 2.0
05         cy = (det.y1 + det.y2) / 2.0
06         if 0 <= cx <= image_w and self.plan_slot_height <= cy <= image_h:
07             detections.append(ClickDetection(det.conf, det.x1, det.y1, det.x2, det.y2))
08     return detections[: self.top_k]
09
10 def _score_siamese(self, plan_crop: np.ndarray, char_crops: list[np.ndarray]) -> np.ndarray:
11     x1_list: list[np.ndarray] = []
12     x2_list: list[np.ndarray] = []
13     for char in char_crops:
14         # Training order is char first, plan second.
15         x1_list.append(_preprocess_siamese(char))
16         x2_list.append(_preprocess_siamese(plan_crop))
17     x1 = np.stack(x1_list, axis=0).astype(np.float32)
18     x2 = np.stack(x2_list, axis=0).astype(np.float32)
19
20     session = self._ensure_siamese_ort()
21     inputs = [inp.name for inp in session.get_inputs()]
22     output = session.get_outputs()[0].name
23     logits = session.run([output], {inputs[0]: x1, inputs[1]: x2})[0]
24     return _sigmoid(logits.reshape(-1))
25
26 @staticmethod
27 def _image_bytes_to_bgr(image_bytes: bytes) -> np.ndarray:
28     image = Image.open(BytesIO(image_bytes)).convert("RGB")
29     return cv2.cvtColor(np.array(image), cv2.COLOR_RGB2BGR)
30
31 def solve(
32     self,
33     target_bytes: bytes,
34     bg_bytes: bytes,
35     return_details: bool = False,
36 ) -> tuple[int, int] | ClickSiameseResult | None:
37     target_bgr = self._image_bytes_to_bgr(target_bytes)
38     bg_bgr = self._image_bytes_to_bgr(bg_bytes)
39     return self.solve_arrays(target_bgr, bg_bgr, return_details=return_details)
40
41 def solve_arrays(
42     self,
43     target_bgr: np.ndarray,
44     bg_bgr: np.ndarray,
45     return_details: bool = False,
46 ) -> tuple[int, int] | ClickSiameseResult | None:
47     plan_crop = self._extract_plan_crop(target_bgr)
48     composed = self._compose_plan_bg(plan_crop, bg_bgr)
49     detections = self._detect_candidates(composed)
50     # Production rule: YOLO must provide exactly four retained candidates.

```

```
01      # Anything else is treated as an unusable captcha and the caller refreshes.
02      if len(detections) != 4:
03          return None
04
05      half = self.char_crop_size // 2
06      char_crops: list[np.ndarray] = []
07      centers: list[tuple[int, int]] = []
08      for det in detections:
09          cx, cy = det.center
10          bx = int(round(cx))
11          by = int(round(cy - self.plan_slot_height))
12          crop = _crop_with_padding(bg_bgr, bx - half, by - half, bx + half, by + half, fill=(128, 128, 128))
13          char_crops.append(crop)
14          centers.append((bx, by))
15
16      sim = self._score_siamese(plan_crop, char_crops)
17      best_idx = int(np.argmax(sim))
18      point = centers[best_idx]
19
20      if not return_details:
21          return point
22      return ClicklSiameseResult(
23          point=point,
24          candidates=centers,
25          candidate_confidences=[det.conf for det in detections],
26          similarity=sim.tolist(),
27          best_index=best_idx,
28      )
29
30
31 _solver: ClicklYolo4SiameseSolver | None = None
32
33
34 def get_clickl_yolo4_siamese_solver(**kwargs: Any) -> ClicklYolo4SiameseSolver:
35     global _solver
36     if _solver is None or kwargs:
37         _solver = ClicklYolo4SiameseSolver(**kwargs)
38     return _solver
39
40
41 def solve_clickl_target_bg(target_bytes: bytes, bg_bytes: bytes) -> tuple[int, int] | bool:
42     """Local captcha API: target+bg bytes -> one bg coordinate, or False."""
43     point = get_clickl_yolo4_siamese_solver().solve(target_bytes, bg_bytes)
44     if point is None:
45         return False
46     return (int(point[0]), int(point[1]))
47
48
49 def preload_clickl_target_bg() -> None:
50     """Load and warm the production YOLO4+Siamese clickl captcha solver before rush time."""
```

```

01     get_clickl_yolo4_siamese_solver().warmup()
02
03 # ===== FILE: core/captcha_yolo4_siamese.py =====
04 from __future__ import annotations
05
06 from dataclasses import dataclass
07 from io import BytesIO
08 from pathlib import Path
09 from typing import Any
10
11 import cv2
12 import numpy as np
13 import onnxruntime as ort
14 from PIL import Image
15
16 from core.yolo_onnx import YoloOnnxPredictor
17
18
19 @dataclass(frozen=True)
20 class Yolo4Detection:
21     conf: float
22     x1: float
23     y1: float
24     x2: float
25     y2: float
26
27     @property
28     def center(self) -> tuple[float, float]:
29         return (self.x1 + self.x2) / 2.0, (self.y1 + self.y2) / 2.0
30
31
32 @dataclass(frozen=True)
33 class Yolo4SiameseResult:
34     points: list[tuple[int, int]]
35     candidates: list[tuple[int, int]]
36     candidate_confidences: list[float]
37     similarity: list[list[float]]
38     matches: list[tuple[int, int]]
39
40
41 def _project_root() -> Path:
42     try:
43         from core.paths import resource_root
44         return resource_root()
45     except Exception:
46         return Path(__file__).resolve().parents[1]
47
48
49 def _resolve_path(path: str | Path) -> Path:
50     p = Path(path)

```



```

01     return p if p.is_absolute() else _project_root() / p
02
03
04 def _sigmoid(x: np.ndarray) -> np.ndarray:
05     return 1.0 / (1.0 + np.exp(-x))
06
07
08 def _crop_with_padding(
09     image: np.ndarray,
10     x1: int,
11     y1: int,
12     x2: int,
13     y2: int,
14     fill: tuple[int, int, int],
15 ) -> np.ndarray:
16     h, w = image.shape[:2]
17     canvas = np.full((max(1, y2 - y1), max(1, x2 - x1), 3), fill, dtype=np.uint8)
18     sx1, sy1 = max(0, x1), max(0, y1)
19     sx2, sy2 = min(w, x2), min(h, y2)
20     if sx2 > sx1 and sy2 > sy1:
21         canvas[sy1 - y1 : sy2 - y1, sx1 - x1 : sx2 - x1] = image[sy1:sy2, sx1:sx2]
22     return canvas
23
24
25 def _preprocess_siamese(image_bgr: np.ndarray, input_size: tuple[int, int] = (112, 112)) -> np.ndarray:
26     rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
27     h, w = input_size
28     ih, iw = rgb.shape[:2]
29     scale = min(w / iw, h / ih)
30     nw, nh = int(iw * scale), int(ih * scale)
31     resized = cv2.resize(rgb, (nw, nh), interpolation=cv2.INTER_CUBIC)
32     canvas = np.full((h, w, 3), 128, dtype=np.uint8)
33     dx, dy = (w - nw) // 2, (h - nh) // 2
34     canvas[dy : dy + nh, dx : dx + nw] = resized
35     arr = canvas.astype(np.float32) / 255.0
36     return np.transpose(arr, (2, 0, 1))
37
38
39 def _optimal_match(sim: np.ndarray) -> list[tuple[int, int]]:
40     from itertools import permutations
41
42     n_rows, n_cols = sim.shape
43     if n_rows == 0 or n_cols == 0 or n_rows > n_cols:
44         return []
45     best_score = -float("inf")
46     best: list[tuple[int, int]] = []
47     for perm in permutations(range(n_cols), n_rows):
48         score = sum(float(sim[row, perm[row]]) for row in range(n_rows))
49         if score > best_score:
50             best_score = score

```

```

01         best = [(row, perm[row]) for row in range(n_rows)]
02     return best
03
04
05 class Yolo4SiameseSolver:
06     """End-to-end click3 solver: target+bg -> YOLO four candidates -> Siamese ordered points."""
07
08     def __init__(
09         self,
10         yolo_path: str | Path | None = None,
11         siamese_path: str | Path | None = None,
12         yolo_imgsz: int = 640,
13         yolo_conf: float = 0.05,
14         yolo_iou: float = 0.45,
15         yolo_device: str = "cpu",
16         siamese_device: str = "cpu",
17         top_k: int = 4,
18         char_crop_size: int = 60,
19         plan_x1: int = 5,
20         plan_x2: int = 25,
21         plan_x3: int = 45,
22         plan_x4: int = 65,
23         plan_y1: int = 5,
24         plan_y2: int = 27,
25         plan_slot_height: int = 44,
26     ) -> None:
27         if yolo_path is None:
28             yolo_path = "core/checkpoints/click3_yolo_plan_bg_char60_best.onnx"
29         if siamese_path is None:
30             siamese_path = "core/checkpoints/click3_siamese_yolo4_posw3_best.onnx"
31
32         self.yolo_path = _resolve_path(yolo_path)
33         self.siamese_path = _resolve_path(siamese_path)
34         self.yolo_imgsz = int(yolo_imgsz)
35         self.yolo_conf = float(yolo_conf)
36         self.yolo_iou = float(yolo_iou)
37         self.top_k = int(top_k)
38         self.char_crop_size = int(char_crop_size)
39         self.plan_xs = [int(plan_x1), int(plan_x2), int(plan_x3), int(plan_x4)]
40         self.plan_y1 = int(plan_y1)
41         self.plan_y2 = int(plan_y2)
42         self.plan_slot_height = int(plan_slot_height)
43
44         self._yolo: YoloOnnxPredictor | None = None
45         self._siamese_ort: ort.InferenceSession | None = None
46
47     def _ensure_yolo(self) -> YoloOnnxPredictor:
48         if self._yolo is None:
49             self._yolo = YoloOnnxPredictor(
50                 self.yolo_path, conf=self.yolo_conf, iou=self.yolo_iou, imgsz=self.yolo_imgsz,

```

```
01         )
02         return self._yolo
03
04     def _ensure_siamese_ort(self) -> ort.InferenceSession:
05         if self._siamese_ort is None:
06             if not self.siamese_path.exists():
07                 raise FileNotFoundError(f"Siamese ONNX not found: {self.siamese_path}")
08             if self.siamese_path.suffix.lower() != ".onnx":
09                 raise ValueError(f"Production Siamese model must be ONNX: {self.siamese_path}")
10             self._siamese_ort = ort.InferenceSession(str(self.siamese_path), providers=["CPUExecutionProvider"])
11         return self._siamese_ort
12
13     def warmup(self) -> None:
14         self._ensure_yolo()
15         self._ensure_siamese_ort()
16         plan_crops = [
17             np.full(
18                 (
19                     max(1, self.plan_y2 - self.plan_y1),
20                     max(1, self.plan_xs[index + 1] - self.plan_xs[index]),
21                     3,
22                 ),
23                 255,
24                 dtype=np.uint8,
25             )
26             for index in range(3)
27         ]
28         bg_bgr = np.full((120, 300, 3), 255, dtype=np.uint8)
29         composed = self._compose_plan_bg(plan_crops, bg_bgr)
30         self._ensure_yolo().predict(composed, max_det=1)
31         char_crops = [
32             np.full((self.char_crop_size, self.char_crop_size, 3), 128, dtype=np.uint8)
33             for _ in range(4)
34         ]
35         self._score_siamese(plan_crops, char_crops)
36
37     def _extract_plan_crops(self, target_bgr: np.ndarray) -> list[np.ndarray]:
38         crops: list[np.ndarray] = []
39         for index in range(3):
40             crops.append(
41                 _crop_with_padding(
42                     target_bgr,
43                     self.plan_xs[index],
44                     self.plan_y1,
45                     self.plan_xs[index + 1],
46                     self.plan_y2,
47                     fill=(255, 255, 255),
48                 )
49             )
50         return crops
```

```
01
02 def _compose_plan_bg(self, plan_crops: list[np.ndarray], bg_bgr: np.ndarray) -> np.ndarray:
03     bg_h, bg_w = bg_bgr.shape[:2]
04     canvas = np.full((self.plan_slot_height + bg_h, bg_w, 3), 255, dtype=np.uint8)
05     slot_w = bg_w // 3
06     for index, crop in enumerate(plan_crops):
07         ch, cw = crop.shape[:2]
08         x = index * slot_w + (slot_w - cw) // 2
09         y = (self.plan_slot_height - ch) // 2
10         canvas[y : y + ch, x : x + cw] = crop
11     canvas[self.plan_slot_height : self.plan_slot_height + bg_h, 0:bg_w] = bg_bgr
12     return canvas
13
14 def _detect_candidates(self, composed_bgr: np.ndarray) -> list[Yolo4Detection]:
15     model = self._ensure_yolo()
16     raw = model.predict(composed_bgr, max_det=300)
17     detections: list[Yolo4Detection] = []
18     image_h, image_w = composed_bgr.shape[:2]
19     for det in raw:
20         cx = (det.x1 + det.x2) / 2.0
21         cy = (det.y1 + det.y2) / 2.0
22         if 0 <= cx <= image_w and self.plan_slot_height <= cy <= image_h:
23             detections.append(Yolo4Detection(det.conf, det.x1, det.y1, det.x2, det.y2))
24     return detections[: self.top_k]
25
26 def _score_siamese(self, plan_crops: list[np.ndarray], char_crops: list[np.ndarray]) -> np.ndarray:
27     x1_list: list[np.ndarray] = []
28     x2_list: list[np.ndarray] = []
29     for plan in plan_crops:
30         for char in char_crops:
31             # Training order is char first, plan second.
32             x1_list.append(_preprocess_siamese(char))
33             x2_list.append(_preprocess_siamese(plan))
34     x1 = np.stack(x1_list, axis=0).astype(np.float32)
35     x2 = np.stack(x2_list, axis=0).astype(np.float32)
36
37     session = self._ensure_siamese_ort()
38     inputs = [inp.name for inp in session.get_inputs()]
39     output = session.get_outputs()[0].name
40     logits = session.run([output], {inputs[0]: x1, inputs[1]: x2})[0]
41     return sigmoid(logits.reshape(len(plan_crops), len(char_crops)))
42
43 @staticmethod
44 def _image_bytes_to_bgr(image_bytes: bytes) -> np.ndarray:
45     image = Image.open(BytesIO(image_bytes)).convert("RGB")
46     return cv2.cvtColor(np.array(image), cv2.COLOR_RGB2BGR)
47
48 def solve(
49     self,
50     target_bytes: bytes,
```

```
01         bg_bytes: bytes,
02         return_details: bool = False,
03     ) -> list[tuple[int, int]] | Yolo4SiameseResult | None:
04         target_bgr = self._image_bytes_to_bgr(target_bytes)
05         bg_bgr = self._image_bytes_to_bgr(bg_bytes)
06         return self.solve_arrays(target_bgr, bg_bgr, return_details=return_details)
07
08     def solve_arrays(
09         self,
10         target_bgr: np.ndarray,
11         bg_bgr: np.ndarray,
12         return_details: bool = False,
13     ) -> list[tuple[int, int]] | Yolo4SiameseResult | None:
14         plan_crops = self._extract_plan_crops(target_bgr)
15         composed = self._compose_plan_bg(plan_crops, bg_bgr)
16         detections = self._detect_candidates(composed)
17         # Production rule: YOLO must provide exactly four retained candidates.
18         # Anything else is treated as an unusable captcha and the caller refreshes.
19         if len(detections) != 4:
20             return None
21
22         half = self.char_crop_size // 2
23         char_crops: list[np.ndarray] = []
24         centers: list[tuple[int, int]] = []
25         for det in detections:
26             cx, cy = det.center
27             bx = int(round(cx))
28             by = int(round(cy - self.plan_slot_height))
29             crop = _crop_with_padding(bg_bgr, bx - half, by - half, bx + half, by + half, fill=(128, 128, 128))
30             char_crops.append(crop)
31             centers.append((bx, by))
32
33         sim = self._score_siamese(plan_crops, char_crops)
34         matches = _optimal_match(sim)
35         if len(matches) < 3:
36             return None
37         points = [centers[char_index] for _plan_index, char_index in matches]
38
39         if not return_details:
40             return points
41         return Yolo4SiameseResult(
42             points=points,
43             candidates=centers,
44             candidate_confidences=[det.conf for det in detections],
45             similarity=sim.tolist(),
46             matches=matches,
47         )
48
49
50     _solver: Yolo4SiameseSolver | None = None
```

```

01
02
03 def get_yolo4_siamese_solver(**kwargs: Any) -> Yolo4SiameseSolver:
04     global _solver
05     if _solver is None or kwargs:
06         _solver = Yolo4SiameseSolver(**kwargs)
07     return _solver
08
09
10 def solve_click3_target_bg(target_bytes: bytes, bg_bytes: bytes) -> list[tuple[int, int]] | bool:
11     """Local captcha API: target+bg bytes -> three ordered bg coordinates, or False."""
12     points = get_yolo4_siamese_solver().solve(target_bytes, bg_bytes)
13     if not points or len(points) != 3:
14         return False
15     return [(int(x), int(y)) for x, y in points]
16
17
18 def preload_click3_target_bg() -> None:
19     """Load and warm the production YOLO4+Siamese captcha solver before rush time."""
20     get_yolo4_siamese_solver().warmup()
21
22 # ===== FILE: core/desktop_notify.py =====
23 # core/desktop_notify.py
24 """
25 桌面通知模块
26
27 用于单账号模式：在需要换座时弹窗通知用户确认。
28
29 支持：
30 - tkinter 消息框（Windows/macOS/Linux 原生）
31 - 日志记录（无GUI环境回退）
32 """
33
34 import os
35 import re
36 import sys
37 import threading
38 import time
39 import queue
40 import tempfile
41 import struct
42 from core.logger import get_logger
43
44 logger = get_logger(__name__)
45 _ICON_PNG_CACHE = {}
46
47
48 def _app_icon_path() -> str:
49     candidates = []
50     base_dir = getattr(sys, "_MEIPASS", "")

```

```

01
02     resume_state = cfg.get("resume_single_state") or {}
03     if isinstance(resume_state, dict) and resume_state.get("active") and resume_state.get("mode") == "single":
04         state_account = str(resume_state.get("account", "")).strip()
05         saved_current = resume_state.get("current_booking") or {}
06         saved_next = resume_state.get("pending_next_slot") or {}
07         if (not state_account or state_account == str(account)) and (saved_current or saved_next):
08             self._emit("[恢复] 检测到上次单账号运行状态，登录后查询当前预约...\n", "#00c8ff")
09             bm = BookingManager(driver, account=account)
10             server_current = None
11             if bm.navigate_to_my_bookings():
12                 server_current = bm.get_current_booking()
13             else:
14                 self._emit(" [WARN] 无法进入我的预约页面，暂不恢复旧任务\n", "#ffab40")
15
16         if server_current and server_current.is_active and server_current.end_time:
17             saved_room = str(saved_current.get("room_name", "")).strip()
18             if saved_room and not server_current.room_name:
19                 server_current.room_name = saved_room
20             if not server_current.room_name:
21                 server_current.room_name = room
22
23         notify_at = resume_state.get("notify_at") or self._notify_at_for_booking(server_current, pre_notify)
24         saved_current_text = (
25             f"{saved_current.get('room_name') or '未知房间'} / 座位{saved_current.get('seat_num') or '?'} / "
26             f"{saved_current.get('start_time') or '?'}-{saved_current.get('end_time') or '?'}"
27         )
28         server_current_text = (
29             f"{server_current.room_name or '未知房间'} / 座位{server_current.seat_num or '?'} / "
30             f"{server_current.start_time or '?'}-{server_current.end_time or '?'}"
31         )
32         saved_next_text = "无"
33         if saved_next:
34             saved_next_text = (
35                 f"{saved_next.get('room_name') or '未知房间'} / 座位{saved_next.get('seat_num') or '?'} / "
36                 f"{saved_next.get('start') or '?'}-{saved_next.get('end') or '?'}"
37             )
38
39         title = "恢复单账号自动换座任务"
40         msg = (
41             "已登录并查询到当前有效预约。 \n\n"
42             f"服务器当前预约:\n {server_current_text}\n\n"
43             f"上次关闭前记录:\n {saved_current_text}\n\n"
44             f"下一次扫描时间:\n {notify_at or '按当前预约结束时间重新计算'} "
45             f"(提前 {pre_notify} 分钟)\n\n"
46             f"上次第二段推荐:\n {saved_next_text}\n\n"
47             "请选择:\n"
48             "继续上次计划: 接管当前预约并继续计时/验证上次推荐\n"
49             "重新扫描: 接管当前预约, 但忽略上次第二段推荐\n"
50             "停止: 保留当前预约, 不继续自动换座"

```

```
01         )
02         choice = notify_resume_choice(title, msg, timeout_seconds=cfg.get("notify_timeout", 300))
03         if choice == "abort":
04             self._emit(" 用户选择不接管旧任务，当前预约保持不变。\\n", "#ffab40")
05             driver.quit()
06             return
07
08         current = BookingInfo(
09             seat_num=server_current.seat_num or saved_current.get("seat_num", ""),
10             room_name=server_current.room_name or saved_current.get("room_name", room),
11             start_time=server_current.start_time or saved_current.get("start_time", ""),
12             end_time=server_current.end_time or saved_current.get("end_time", ""),
13             status=server_current.status or "有效",
14         )
15         resume_pending_slot = saved_next if choice == "resume" and saved_next else None
16         resume_force_scan_once = bool(choice == "rescan" or resume_pending_slot)
17         notify_at = notify_at or self._notify_at_for_booking(current, pre_notify)
18         self._set_single_runtime_state(
19             "waiting_next_scan",
20             current=current,
21             notify_at=notify_at,
22             reason="resume_takeover",
23             extra={"resumed_from_saved_state": True},
24         )
25
26         if not bm.return_to_seat_grid():
27             self._emit(f" 返回座位图失败，尝试进入当前预约房间: {current.room_name}\\n", "#ffab40")
28         if current.room_name and not enter_room(driver, campus, current.room_name, account=account):
29             self._emit(" [WARN] 无法进入当前预约房间，后续换座前会再次尝试\\n", "#ffab40")
30         self._emit(
31             f" [OK] 已接管当前预约: 座位 {current.seat_num} [{current.room_name}] "
32             f"{current.start_time}-{current.end_time}\\n\\n",
33             "#00e676",
34         )
35     else:
36         self._emit(" 未查询到有效预约，按当前配置重新开始第一段预约。\\n", "#ffab40")
37
38     if current is None:
39         # [2] API 扫描并生成候选方案
40         self._emit("[2] API 扫描空闲座位时间段（不点击具体座位）...\\n", "#7c5cfc")
41         scan_progress_window = ScanProgressWindow("API 单账号扫描进度").start()
42         try:
43             first_plan = build_api_plan(
44                 client,
45                 campus=campus,
46                 target_room=room,
47                 day_start=day_start,
48                 day_end=day_end,
49                 date=cfg.get("date", ""),
50                 cross_room=cross_room,
```



```

01         cross_room_rooms=self._cross_room_candidates(cfg, campus, room),
02         preferred_seats=cfg.get("preferred_seats", {}),
03         priority_mode=cfg.get("priority_mode", "longest_first"),
04         accounts_count=len(accounts),
05         cross_room_min_gain_minutes=cfg.get("cross_room_min_gain_minutes", 0),
06         include_intervals=cfg.get("api_report_include_intervals", True),
07         api_scan_workers=cfg.get("api_scan_workers", 10),
08         stop_event=self._stop_event,
09         progress=progress,
10     )
11 finally:
12     scan_progress_window.close()
13     scan_progress_window = None
14     report_path = save_plan_report(first_plan, prefix="api_booking_plan")
15     self._emit("\n" + format_plan_summary(first_plan, dry_run=False) + "\n", "#00e676" if first_plan.get("success") else "#ff5252")
16     self._emit(f"API方案报告: {report_path}\n", "#00c8ff")
17
18     final = (first_plan.get("selection") or {}).get("final_recommendation")
19     if not first_plan.get("success") or not final:
20         self._emit("[FAIL] API 未找到可预约方案, 本次不回退逐座位点击扫描.\n", "#ff5252")
21         driver.quit()
22         return
23
24     title, msg, choices = self._build_api_booking_options_notification(first_plan)
25     option_items = self._plan_selectable_options(first_plan)
26     selected_id = self._select_seat_option(
27         title,
28         msg,
29         option_items,
30         "abort",
31         cfg.get("notify_timeout", 300),
32     )
33     if selected_id == "abort":
34         self._emit(" 用户取消预约, 本次任务结束\n", "#ffab40")
35         driver.quit()
36         return
37
38     selected_final = self._option_by_id(first_plan, selected_id) or final
39     first_slot = self._api_item_to_slot(selected_final)
40     self._emit(f" 第一阶段: 座位{first_slot.seat_num} [{first_slot.room_name}] {first_slot.start}-{first_slot.end}\n", "#00e676")
41
42     if first_slot.room_name != room:
43         self._emit(f" 切换到推荐房间: {first_slot.room_name}\n", "#7c5cfc")
44         if not enter_room(driver, campus, first_slot.room_name, account=account):
45             self._emit("[FAIL] 进入推荐房间失败\n", "#ff5252")
46             driver.quit()
47             return
48
49     if not self._book_seat(booker, first_slot.seat_num, self._booking_start(first_slot), self._booking_end(first_slot)):
50         self._emit("[FAIL] 第一阶段预约失败\n", "#ff5252")

```

```
01         driver.quit()
02         return
03
04         current = BookingInfo(
05             seat_num=first_slot.seat_num,
06             room_name=first_slot.room_name,
07             start_time=first_slot.start,
08             end_time=first_slot.end,
09             status="有效",
10         )
11         self._emit(f" [OK] 预约成功!\n\n", "#00e676")
12
13         # 发送第一段成功邮件
14         try:
15             title, content = build_success_email(account, current.room_name, current.seat_num, current.start_time, current.end_time)
16             send_email(title, content)
17         except Exception:
18             pass
19
20         upcoming_preview = build_single_followup_preview(
21             first_plan,
22             current.end_time,
23             day_end,
24             campus=campus,
25             current_room=current.room_name,
26             preferred_seats=cfg.get("preferred_seats", {}),
27             priority_mode=cfg.get("priority_mode", "longest_first"),
28             cross_room=cross_room,
29             cross_room_min_gain_minutes=cfg.get("cross_room_min_gain_minutes", 0),
30             max_segments=2,
31         )
32     else:
33         upcoming_preview = []
34
35     self._set_single_runtime_state(
36         "waiting_next_scan",
37         current=current,
38         notify_at=self._notify_at_for_booking(current, pre_notify),
39         reason="single_loop_started",
40         extra={"upcoming_slots": [self._slot_to_state(slot) for slot in upcoming_preview]},
41     )
42
43     # [3] 持续循环
44     loop_num = 1
45     while not self._stop_event.is_set():
46         loop_num += 1
47         current_end_mins = _time_to_minutes(current.end_time)
48         day_end_mins = _time_to_minutes(day_end)
49
50         if current_end_mins >= day_end_mins:
```

```
01         self._emit(f"已覆盖全天目标 {day_start}-{day_end}, 结束。\\n", "#00e676")
02         self._set_single_runtime_state("completed", current=current, active=False, reason="covered_target_range")
03         self._clear_single_runtime_state()
04         break
05
06     # 计算等待时间
07     notify_at_mins = current_end_mins - pre_notify
08     notify_at_text = _minutes_to_time(notify_at_mins) if notify_at_mins >= 0 else ""
09     self._set_single_runtime_state(
10         "waiting_next_scan",
11         current=current,
12         notify_at=notify_at_text,
13         reason="waiting_next_scan",
14         extra={"upcoming_slots": [self._slot_to_state(slot) for slot in upcoming_preview]},
15     )
16     now = _bj_now()
17     now_mins = now.hour * 60 + now.minute
18
19     force_scan_now = bool(resume_force_scan_once)
20     resume_force_scan_once = False
21     wait_minutes = notify_at_mins - now_mins
22     if force_scan_now:
23         self._emit(f"恢复模式: 立即重新扫描并验证上次信息...\\n", "#ffab40")
24     elif wait_minutes > 0:
25         notify_h = notify_at_mins // 60
26         notify_m = notify_at_mins % 60
27         self._emit(f"下一次扫描: {notify_h:02d}:{notify_m:02d} (距结束 {pre_notify} 分钟前)\\n", "#8888aa")
28
29     # 分段等待, 每秒检查 stop_event
30     waited = 0
31     target_wait = wait_minutes * 60
32     while waited < target_wait:
33         if self._stop_event.is_set():
34             driver.quit()
35             return
36         sleep_chunk = min(1, target_wait - waited)
37         _time.sleep(sleep_chunk)
38         waited += sleep_chunk
39     else:
40         self._emit(f"提醒时间已过, 立即扫描下一阶段...\\n", "#ffab40")
41
42     if self._stop_event.is_set():
43         break
44
45     # 重新扫描
46     self._emit(f"\\n[阶段{loop_num}] API 寻找 {current.end_time} 之后的座位...\\n", "#7c5cfc")
47     scan_preferred_seats = cfg.get("preferred_seats", {})
48     scan_priority_mode = cfg.get("priority_mode", "longest_first")
49     if resume_pending_slot:
50         scan_preferred_seats = self._preferred_with_resume_slot(scan_preferred_seats, campus, resume_pending_slot)
```

```
01         scan_priority_mode = "prefer_first"
02         self._emit(
03             f" 恢复模式: 优先验证上次推荐 座位 {resume_pending_slot.get('seat_num')} "
04             f"[{resume_pending_slot.get('room_name')}] \n",
05             "#ffab40",
06         )
07         scan_progress_window = ScanProgressWindow("API 下一段扫描进度").start()
08         try:
09             next_plan = build_api_plan(
10                 client,
11                 campus=campus,
12                 target_room=current.room_name,
13                 day_start=current.end_time,
14                 day_end=day_end,
15                 date=cfg.get("date", ""),
16                 cross_room=cross_room,
17                 cross_room_rooms=self._cross_room_candidates(cfg, campus, current.room_name),
18                 preferred_seats=scan_preferred_seats,
19                 priority_mode=scan_priority_mode,
20                 accounts_count=len(accounts),
21                 cross_room_min_gain_minutes=cfg.get("cross_room_min_gain_minutes", 0),
22                 include_intervals=cfg.get("api_report_include_intervals", True),
23                 api_scan_workers=cfg.get("api_scan_workers", 10),
24                 stop_event=self._stop_event,
25                 progress=progress,
26             )
27         finally:
28             scan_progress_window.close()
29             scan_progress_window = None
30             resume_pending_slot = None
31             report_path = save_plan_report(next_plan, prefix="api_booking_plan")
32             self._emit("\n" + format_plan_summary(next_plan, dry_run=False) + "\n", "#00e676" if next_plan.get("success") else "#ffab40")
33             self._emit(f"API方案报告: {report_path} \n", "#00c8ff")
34
35             next_final = (next_plan.get("selection") or {}).get("final_recommendation")
36             if not next_plan.get("success") or not next_final:
37                 self._emit("[WARN] API 未找到可用座位\n", "#ffab40")
38                 self._set_single_runtime_state(
39                     "no_next_seat",
40                     current=current,
41                     notify_at=notify_at_text,
42                     active=False,
43                     reason="api_no_next_seat",
44                     extra={"last_report_path": report_path},
45                 )
46                 title, msg = build_no_seat_notification(current.end_time, day_end)
47                 notify_and_confirm(title, msg, timeout_seconds=120)
48                 self._clear_single_runtime_state()
49             break
50
```

```
01         recommended_slot = self._api_item_to_slot(next_final)
02         recommended_same_room = recommended_slot.room_name == current.room_name
03         self._emit(
04             f" 推荐: 座位{recommended_slot.seat_num} [{recommended_slot.room_name}] "
05             f"{recommended_slot.start}-{recommended_slot.end} "
06             f"({'同房间' if recommended_same_room else '跨房间'})\n",
07             "#00e676",
08         )
09         self._set_single_runtime_state(
10             "pending_change_confirmation",
11             current=current,
12             pending_next_slot=recommended_slot,
13             notify_at=notify_at_text,
14             reason="next_slot_found",
15             extra={"last_report_path": report_path},
16         )
17
18         # 通知用户
19         if auto_cancel:
20             confirmed = True
21             next_slot = recommended_slot
22             self._emit(" 自动模式: 无需用户确认\n", "#8888aa")
23         else:
24             title, msg, choices = self._build_next_seat_options_notification(current, next_plan)
25             option_items = self._plan_selectable_options(next_plan)
26             selected_id = self._select_seat_option(
27                 title,
28                 msg,
29                 option_items,
30                 "abort",
31                 cfg.get("notify_timeout", 300),
32             )
33             confirmed = selected_id != "abort"
34             selected_next = self._option_by_id(next_plan, selected_id) if confirmed else None
35             next_slot = self._api_item_to_slot(selected_next or next_final)
36
37         if not confirmed:
38             self._emit(" 用户拒绝换座, 结束\n", "#ffab40")
39             self._set_single_runtime_state(
40                 "user_declined_change",
41                 current=current,
42                 pending_next_slot=next_slot,
43                 active=False,
44                 reason="user_declined_change",
45             )
46             self._clear_single_runtime_state()
47             break
48
49         same_room = next_slot.room_name == current.room_name
50         self._emit(
```

```
01         f" 已选择下一段: 座位{next_slot.seat_num} [{next_slot.room_name}] "
02         f"{next_slot.start}-{next_slot.end} ({' 同房间' if same_room else '跨房间'})\n",
03         "#00e676",
04     )
05     self._set_single_runtime_state(
06         "pending_change_confirmation",
07         current=current,
08         pending_next_slot=next_slot,
09         notify_at=notify_at_text,
10         reason="next_slot_selected",
11         extra={"last_report_path": report_path},
12     )
13
14     # 执行换座
15     self._emit(" 执行换座...\n", "#7c5cfc")
16     self._set_single_runtime_state(
17         "changing",
18         current=current,
19         pending_next_slot=next_slot,
20         reason="change_confirmed",
21     )
22     bm = BookingManager(driver, account=account)
23
24     if not bm.navigate_to_my_bookings():
25         self._emit(" [FAIL] 无法进入我的预约页面\n", "#ff5252")
26         break
27     if not bm.cancel_booking():
28         self._emit(" [FAIL] 取消预约失败\n", "#ff5252")
29         break
30     self._set_single_runtime_state(
31         "current_cancelled_booking_next",
32         current=current,
33         pending_next_slot=next_slot,
34         reason="current_cancelled",
35     )
36     if not bm.return_to_seat_grid():
37         self._emit(" [FAIL] 返回座位图失败\n", "#ff5252")
38         break
39
40     # 跨房间切换
41     if next_slot.room_name != current.room_name:
42         if not enter_room(driver, campus, next_slot.room_name, account=account):
43             self._emit(" [FAIL] 进入新房间失败\n", "#ff5252")
44             break
45
46     # 预约新座位
47     if not self._book_seat(booker, next_slot.seat_num, self._booking_start(next_slot), self._booking_end(next_slot)):
48         self._emit(" [FAIL] 新座位预约失败\n", "#ff5252")
49         self._set_single_runtime_state(
50             "new_booking_failed_after_cancel",
```

```

01         current=current,
02         pending_next_slot=next_slot,
03         reason="new_booking_failed_after_cancel",
04     )
05     break
06
07     current = BookingInfo(
08         seat_num=next_slot.seat_num,
09         room_name=next_slot.room_name,
10         start_time=next_slot.start,
11         end_time=next_slot.end,
12         status="有效",
13     )
14     self._emit(f" [OK] 换座成功! 座位{current.seat_num} {current.start_time}-{current.end_time}\n\n", "#00e676")
15     upcoming_preview = build_single_followup_preview(
16         next_plan,
17         current.end_time,
18         day_end,
19         campus=campus,
20         current_room=current.room_name,
21         preferred_seats=cfg.get("preferred_seats", {}),
22         priority_mode=cfg.get("priority_mode", "longest_first"),
23         cross_room=cross_room,
24         cross_room_min_gain_minutes=cfg.get("cross_room_min_gain_minutes", 0),
25         max_segments=2,
26     )
27     self._set_single_runtime_state(
28         "waiting_next_scan",
29         current=current,
30         notify_at=self._notify_at_for_booking(current, pre_notify),
31         reason="change_success",
32         extra={"upcoming_slots": [self._slot_to_state(slot) for slot in upcoming_preview]},
33     )
34
35     # 发送邮件
36     try:
37         title, content = build_success_email(account, current.room_name, current.seat_num, current.start_time, current.end_time)
38         send_email(title, content)
39     except Exception:
40         pass
41
42     # 结束
43     try:
44         driver.quit()
45     except Exception:
46         pass
47     self._clear_active_driver(driver)
48     self._emit(f"\n{'='*40}\n单账号模式结束\n", "#00c8ff")
49
50     def _build_api_booking_notification(self, plan: dict) -> tuple:

```

```
01 """构建 API 推荐后的首次预约确认弹窗。"""
02 cfg = plan.get("config", {}) or {}
03 selection = plan.get("selection", {}) or {}
04
05 def fmt(item):
06     if not item:
07         return "无"
08     return (
09         f"{item.get('room_name')} / 座位{item.get('seat_num')} / "
10         f"{item.get('start')}-{item.get('end')} / "
11         f"{item.get('duration_minutes')}分钟"
12     )
13
14 final = selection.get("final_recommendation")
15 title = "座位预约确认 - API 已生成推荐方案"
16 message = (
17     f"目标: {cfg.get('campus', '')} / {cfg.get('target_room', '')}\n"
18     f"有效时段: {cfg.get('effective_range', '')}\n\n"
19     f"当前房间规则选择:\n"
20     f"    {fmt(selection.get('current_room_choice'))}\n\n"
21     f"当前房间最长时段:\n"
22     f"    {fmt(selection.get('current_room_best'))}\n\n"
23 )
24 if cfg.get("cross_room"):
25     message += (
26         f"跨房间最佳候选:\n"
27         f"    {fmt(selection.get('cross_room_best'))}\n\n"
28     )
29 message += (
30     f"最终推荐:\n"
31     f"    {fmt(final)}\n\n"
32     f"决策说明: {selection.get('decision_note', '')}\n\n"
33     "点击「是」: 预约最终推荐座位\n"
34     "点击「否」: 不预约并结束本次任务"
35 )
36 return title, message
37
38 def _plan_selectable_options(self, plan: dict) -> list:
39     selection = plan.get("selection", {}) or {}
40     cfg = plan.get("config", {}) or {}
41     target_room = cfg.get("target_room", "")
42     options = list(selection.get("selectable_options") or [])
43     final = selection.get("final_recommendation")
44     if not options and final:
45         options = [dict(final, option_id="opt1", title="最终推荐", reason=selection.get("decision_note", ""), is_recommended=True)]
46
47     normalized = []
48     for idx, option in enumerate(options, start=1):
49         item = dict(option)
50         item.setdefault("option_id", f"opt{idx}")
```



```

01         item.setdefault("title", f"候选{idx}")
02         item.setdefault("target_room", target_room)
03         normalized.append(item)
04     return normalized
05
06     def _option_by_id(self, plan: dict, option_id: str):
07         for option in self._plan_selectable_options(plan):
08             if option.get("option_id") == option_id:
09                 return option
10         return None
11
12     def _format_option_line(self, idx: int, item: dict) -> str:
13         marker = " [推荐]" if item.get("is_recommended") else ""
14         return (
15             f"{idx}. {item.get('title', '候选')}{marker}: "
16             f"{item.get('room_name')} / 座位{item.get('seat_num')} / "
17             f"{item.get('start')}-{item.get('end')} / "
18             f"{item.get('duration_minutes', 0)}分钟"
19         )
20
21     def _build_api_booking_options_notification(self, plan: dict) -> tuple:
22         cfg = plan.get("config", {}) or {}
23         options = self._plan_selectable_options(plan)
24         title = "? 座位预约选择"
25         lines = [
26             f"? {cfg.get('campus', '')} / {cfg.get('target_room', '')}",
27             f"      {cfg.get('effective_range', '')}",
28             "? 从列表里选一个候选座位，确认后再执行预约。",
29         ]
30         choices = [(option.get("option_id"), f"候选{idx}") for idx, option in enumerate(options, start=1)]
31         choices.append(("abort", "取消"))
32         return title, "\n".join(lines), choices
33         """构建首次预约候选选择弹窗。"""
34         cfg = plan.get("config", {}) or {}
35         selection = plan.get("selection", {}) or {}
36         options = self._plan_selectable_options(plan)
37         title = "座位预约选择 - API 已生成候选方案"
38         lines = [
39             f"目标: {cfg.get('campus', '')} / {cfg.get('target_room', '')}",
40             f"有效时段: {cfg.get('effective_range', '')}",
41             f"决策说明: {selection.get('decision_note', '')}",
42             "",
43             "请选择要预约的座位:",
44         ]
45         for idx, option in enumerate(options, start=1):
46             lines.append(self._format_option_line(idx, option))
47             reason = option.get("reason")
48             if reason:
49                 lines.append(f"    说明: {reason}")
50         lines.extend(["", "点击对应按钮预约该候选；点击取消则结束本次任务。"])

```

```

01     choices = [(option.get("option_id"), f"选{idx}") for idx, option in enumerate(options, start=1)]
02     choices.append(("abort", "取消"))
03     return title, "\n".join(lines), choices
04
05 def _build_next_seat_options_notification(self, current, plan: dict) -> tuple:
06     cfg = plan.get("config", {}) or {}
07     options = self._plan_selectable_options(plan)
08     title = "? 换座候选选择"
09     lines = [
10         f"当前: {current.room_name} / 座位{current.seat_num} / {current.start_time}-{current.end_time}",
11         f"下一段范围: {cfg.get('effective_range', '')}",
12         "? 选择下一段座位; 取消则保持当前预约。",
13     ]
14     choices = [(option.get("option_id"), f"候选{idx}") for idx, option in enumerate(options, start=1)]
15     choices.append(("abort", "取消"))
16     return title, "\n".join(lines), choices
17     """构建单账号下一段候选选择弹窗。"""
18     cfg = plan.get("config", {}) or {}
19     options = self._plan_selectable_options(plan)
20     title = "换座提醒 - 请选择下一阶段座位"
21     lines = [
22         f"当前预约: {current.room_name} / 座位{current.seat_num} / {current.start_time}-{current.end_time}",
23         f"下一段扫描范围: {cfg.get('effective_range', '')}",
24         "",
25         "请选择下一阶段座位:",
26     ]
27     for idx, option in enumerate(options, start=1):
28         lines.append(self._format_option_line(idx, option))
29         reason = option.get("reason")
30         if reason:
31             lines.append(f"    说明: {reason}")
32     lines.extend([
33         "",
34         "点击对应按钮: 取消当前预约并预约该候选",
35         "点击取消: 保持当前座位, 不换座",
36     ])
37     choices = [(option.get("option_id"), f"选{idx}") for idx, option in enumerate(options, start=1)]
38     choices.append(("abort", "取消"))
39     return title, "\n".join(lines), choices
40
41 def _build_multi_booking_options_notification(self, plan: dict, schedule_options: list, accounts: list) -> tuple:
42     cfg = plan.get("config", {}) or {}
43     title = "? 多账号方案选择"
44     pool_count = 0
45     if schedule_options:
46         pool_count = int(schedule_options[0].get("candidate_pool_count") or len(schedule_options))
47     lines = [
48         f"? {cfg.get('campus', '')} / {cfg.get('target_room', '')}",
49         f"    {cfg.get('effective_range', '')}    ? {len(accounts)} 个账号",
50         f"? 已生成 {pool_count or len(schedule_options)} 套候选, 精选 {len(schedule_options)} 套展示。",

```

```

01         "",
02         "? 从列表中选择一套完整方案；确认后依次登录账号并预约。",
03     ]
04     choices = [(option.get("option_id"), f"方案{idx}") for idx, option in enumerate(schedule_options, start=1)]
05     choices.append(("abort", "取消"))
06     return title, "\n".join(lines), choices
07     """构建多账号多套方案选择弹窗。"""
08     cfg = plan.get("config", {}) or {}
09     title = "多账号预约选择 - API 已生成多套方案"
10     lines = [
11         f"目标: {cfg.get('campus', '')} / {cfg.get('target_room', '')}",
12         f"有效时段: {cfg.get('effective_range', '')}",
13         f"可用账号数: {len(accounts)}",
14         "",
15         "请选择一套完整方案:",
16     ]
17
18     for idx, option in enumerate(schedule_options, start=1):
19         schedule = option.get("schedule", {}) or {}
20         lines.append(f"方案{idx}: {option.get('title', '')}")
21         lines.append(f"    {option.get('description', '')}")
22         note = option.get("note")
23         if note:
24             lines.append(f"    说明: {note}")
25         for seg_idx, seg in enumerate(schedule.get("segments", []) or [], start=1):
26             lines.append(
27                 f"    账号{seg_idx}: {seg.get('room_name')} / 座位{seg.get('seat_num')} / "
28                 f"{seg.get('start')}-{seg.get('end')} / {seg.get('duration_minutes')}分钟"
29             )
30         gaps = schedule.get("gaps", []) or []
31         if gaps:
32             gap_text = ", ".join(f"{gap.get('start')}-{gap.get('end')}" for gap in gaps)
33             lines.append(f"    未覆盖: {gap_text}")
34         lines.append("")
35
36     lines.append("点击对应按钮后，程序会按所选方案依次登录账号并预约。")
37     choices = [(option.get("option_id"), f"方案{idx}") for idx, option in enumerate(schedule_options, start=1)]
38     choices.append(("abort", "取消"))
39     return title, "\n".join(lines), choices
40
41 def _build_multi_booking_notification(self, plan: dict, schedule: dict, accounts: list) -> tuple:
42     """构建多账号 API 分段预约确认弹窗。"""
43     cfg = plan.get("config", {}) or {}
44     title = "多账号预约确认 - API 已生成分段方案"
45     lines = [
46         f"目标: {cfg.get('campus', '')} / {cfg.get('target_room', '')}",
47         f"有效时段: {cfg.get('effective_range', '')}",
48         f"账号数: {len(accounts)}",
49         f"分段数: {schedule.get('segment_count', 0)}",
50         f"覆盖: {schedule.get('total_covered_minutes', 0)} / {schedule.get('desired_minutes', 0)} 分钟",

```

```
01         "",
02         "分段方案:",
03     ]
04
05     for idx, seg in enumerate(schedule.get("segments", []) or [], start=1):
06         account_label = f"账号{idx}"
07         if idx <= len(accounts):
08             account_label = f"账号{idx}"
09         lines.append(
10             f"    {account_label}: {seg.get('room_name')} / 座位{seg.get('seat_num')} / "
11             f"{seg.get('start')}-{seg.get('end')} / {seg.get('duration_minutes')}分钟"
12         )
13
14     gaps = schedule.get("gaps", []) or []
15     if gaps:
16         lines.extend(["", "未覆盖时段:"])
17         for gap in gaps:
18             lines.append(f"    {gap.get('start')}-{gap.get('end')}")
19
20     lines.extend([
21         "",
22         "点击「是」: 按上述分段依次登录账号并预约",
23         "点击「否」: 不预约并结束本次任务",
24     ])
25     return title, "\n".join(lines)
26
27 # -----
28 # 工具方法: 预约座位 (含验证码循环)
29 # -----
30 def _book_seat(self, booker, seat_num, start, end) -> bool:
31     """预约单个座位, 含验证码重试循环。返回 True/False。"""
32     booker.current_seat = str(seat_num)
33     booker.current_retry = 0
34     if not booker.select_time_and_wait(seat_num, start, end):
35         return False
36     if not booker.fire_submit_trigger():
37         booker.close_popup()
38         return False
39
40     for retry in range(1, booker.get_captcha_max_retries() + 1):
41         if self._stop_event.is_set():
42             return False
43         booker.current_retry = retry
44         solve_data = booker.pre_solve_captcha()
45         if solve_data.get("no_captcha"):
46             result = booker.check_result()
47             return result.get("status") == "success"
48         if not solve_data.get("solved"):
49             booker._refresh_click_captcha(previous_key=solve_data.get("captcha_key", ""), wait_timeout=1.0)
50             continue
```

```

01         if booker.fire_captcha_blitz(solve_data):
02             result = booker.check_result()
03             if result.get("status") == "success":
04                 return True
05             if result.get("status") in ("stop", "blacklist"):
06                 return False
07             if result.get("status") == "retry_captcha":
08                 booker.last_captcha_auto_refreshed = True
09         if booker.last_captcha_auto_refreshed:
10             booker.last_captcha_auto_refreshed = False
11             continue
12         if not booker.is_captcha_popup_present():
13             result = booker.check_result()
14             if result.get("status") == "success":
15                 return True
16             if result.get("status") == "retry_captcha":
17                 continue
18             if result.get("status") in ("stop", "blacklist"):
19                 return False
20             booker._close_captcha_modal()
21             booker.close_popup()
22             return False
23         booker._refresh_click_captcha(previous_key=solve_data.get("captcha_key", ""), wait_timeout=1.0)
24     return False
25
26 # ===== FILE: build.py =====
27 # -*- coding: utf-8 -*-
28 from __future__ import annotations
29
30 import argparse
31 import json
32 import os
33 import shutil
34 import subprocess
35 import sys
36 from pathlib import Path
37 from zipfile import ZIP_DEFLATED, ZipFile
38
39 ROOT = Path(__file__).resolve().parent
40 APP_NAME = "LNU-LibSeat"
41 APP_VERSION = "v2.6.0"
42 APP_ICON = "OIP-C.ico"
43 DIST_NAME = f"{APP_NAME}-{APP_VERSION}"
44 DIST_ROOT = ROOT / "dist"
45 DIST_DIR = DIST_ROOT / DIST_NAME
46 BUILD_WORK = ROOT / "build" / "pyinstaller"
47 INVALID_PACKAGE_NAME_CHARS = set('<>:/\\|?*')
48
49 REQUIRED_PACKAGES = [
50     "pyinstaller",

```

```

01     "selenium">=4.20.0",
02     "webdriver-manager">=4.0.2",
03     "requests">=2.31.0",
04     "Pillow">=10.1.0",
05     "ddddocr",
06     "PySide6">=6.7.0",
07     "mss">=9.0.1",
08     "opencv-python">=4.8.0",
09     "numpy">=1.24.0",
10     "onnxruntime">=1.17.0",
11 ]
12
13 EXCLUDE_MODULES = [
14     "torch",
15     "torchvision",
16     "timm",
17     "scipy",
18     "matplotlib",
19     "pandas",
20     "ultralitics",
21     "tensorboard",
22     "tensorflow",
23 ]
24
25 CLEAN_CONFIG = {
26     "accounts": [],
27     "campus": "崇山校区图书馆",
28     "room": "三楼智慧研修空间",
29     "day_start": "09:00",
30     "day_end": "22:00",
31     "mode": "single",
32     "dry_run": False,
33     "api_scan_workers": 10,
34     "cross_room": False,
35     "cross_room_rooms": {},
36     "receiver_email": "",
37     "pre_notify": 30,
38     "auto_cancel": False,
39     "priority_mode": "longest_first",
40     "preferred_seats": {},
41     "theme": "auto",
42     "low_animation": False,
43 }
44
45
46 def run(cmd: list[str], *, cwd: Path | None = None) -> None:
47     print("[run]", " ".join(str(part) for part in cmd))
48     subprocess.check_call(cmd, cwd=str(cwd or ROOT))
49
50

```

```

01 def default_dist_name(app_name: str, app_version: str) -> str:
02     return f"{app_name}-{app_version}" if app_version else app_name
03
04
05 def validate_package_name(value: str, label: str) -> str:
06     value = value.strip()
07     if not value:
08         raise ValueError(f"{label} cannot be empty.")
09     invalid = sorted(char for char in value if char in INVALID_PACKAGE_NAME_CHARS)
10     if invalid:
11         raise ValueError(f"{label} contains invalid filename characters: { ' '.join(invalid)}")
12     if value.endswith((".", " ")):
13         raise ValueError(f"{label} cannot end with a dot or space.")
14     return value
15
16
17 def configure_package_names(app_name: str, app_version: str, dist_name: str | None) -> None:
18     global APP_NAME, APP_VERSION, DIST_NAME, DIST_DIR
19
20     APP_NAME = validate_package_name(app_name, "--app-name")
21     APP_VERSION = app_version.strip()
22     if APP_VERSION:
23         APP_VERSION = validate_package_name(APP_VERSION, "--app-version")
24     DIST_NAME = validate_package_name(
25         default_dist_name(APP_NAME, APP_VERSION) if dist_name is None else dist_name,
26         "--dist-name",
27     )
28     DIST_DIR = DIST_ROOT / DIST_NAME
29
30
31 def build_python() -> Path:
32     if sys.platform == "win32":
33         venv_python = ROOT / "env" / "Scripts" / "python.exe"
34     else:
35         venv_python = ROOT / "env" / "bin" / "python"
36     if venv_python.exists():
37         return venv_python
38     return Path(sys.executable)
39
40
41 def ensure_build_deps(py: Path, index_url: str | None) -> None:
42     cmd = [str(py), "-m", "pip", "install", "--upgrade"] + REQUIRED_PACKAGES
43     if index_url:
44         cmd += ["-i", index_url]
45     run(cmd)
46
47
48 def add_data_arg(src: Path, dst: str) -> str:
49     sep = ";" if sys.platform == "win32" else ":"
50     return f"{src}{sep}{dst}"

```

```

01
02
03 def validate_icon_file() -> None:
04     icon_path = ROOT / APP_ICON
05     if not icon_path.exists():
06         raise FileNotFoundError(f"App icon not found: {icon_path}")
07
08     with icon_path.open("rb") as f:
09         header = f.read(6)
10         if len(header) != 6 or header[:4] != b"\x00\x00\x01\x00":
11             raise ValueError(f"App icon is not a valid .ico file: {icon_path}")
12         if int.from_bytes(header[4:6], "little") < 1:
13             raise ValueError(f"App icon contains no icon images: {icon_path}")
14
15
16 def pyinstaller_cmd(py: Path) -> list[str]:
17     cmd = [
18         str(py),
19         "-m",
20         "PyInstaller",
21         "--noconfirm",
22         "--clean",
23         "--onedir",
24         "--windowed",
25         "--name",
26         APP_NAME,
27         "--distpath",
28         str(DIST_ROOT),
29         "--workpath",
30         str(BUILD_WORK),
31         "--specpath",
32         str(BUILD_WORK),
33         "--icon",
34         str(ROOT / APP_ICON),
35         "--add-data",
36         add_data_arg(ROOT / APP_ICON, "."),
37         "--collect-all",
38         "ddddocr",
39         "--collect-all",
40         "onnxruntime",
41         "--collect-all",
42         "cv2",
43         "--collect-all",
44         "numpy",
45         "--collect-submodules",
46         "selenium",
47         "--collect-submodules",
48         "webdriver_manager",
49         "--collect-submodules",
50         "mss",

```



```

01     "--collect-data",
02     "certifi",
03     "--hidden-import",
04     "tkinter",
05 ]
06 for model in sorted((ROOT / "core" / "checkpoints").glob("*.onnx")):
07     cmd += ["--add-data", add_data_arg(model, "core/checkpoints")]
08 for module in EXCLUDE_MODULES:
09     cmd += ["--exclude-module", module]
10 cmd.append(str(ROOT / "app.py"))
11 return cmd
12
13
14 def copy_tree(src: Path, dst: Path) -> None:
15     if not src.exists():
16         return
17     if dst.exists():
18         shutil.rmtree(dst)
19     shutil.copytree(src, dst)
20
21
22 def write_runtime_files() -> None:
23     DIST_DIR.mkdir(parents=True, exist_ok=True)
24
25     icon_src = ROOT / APP_ICON
26     if icon_src.exists():
27         shutil.copy2(icon_src, DIST_DIR / APP_ICON)
28
29     copy_tree(ROOT / "info", DIST_DIR / "info")
30     (DIST_DIR / "logs").mkdir(exist_ok=True)
31
32     with open(DIST_DIR / "config_data.json", "w", encoding="utf-8") as f:
33         json.dump(CLEAN_CONFIG, f, ensure_ascii=False, indent=2)
34
35     readme = """LNU-LibSeat 使用说明
36
37 1. 双击 LNU-LibSeat.exe 启动主程序。
38 2. 请整文件夹一起压缩和发送，不要只移动 exe；_internal 是运行依赖目录。
39 3. config_data.json 会保存用户填写的账号、房间、邮箱和偏好；首次分发时已留空账号信息。
40 4. logs 目录保存运行日志和 API 方案报告。
41 5. 目标电脑需要安装 Microsoft Edge。首次运行如需下载匹配的浏览器驱动，需要联网。
42 6. 邮箱栏留空时不会发送邮件。
43 """
44     with open(DIST_DIR / "使用说明.txt", "w", encoding="utf-8") as f:
45         f.write(readme)
46
47
48 def clean_previous_output() -> None:
49     if DIST_DIR.exists():
50         shutil.rmtree(DIST_DIR)

```

```

01     old = DIST_ROOT / APP_NAME
02     if old.exists():
03         shutil.rmtree(old)
04     old_build = BUILD_WORK / APP_NAME
05     if old_build.exists():
06         shutil.rmtree(old_build)
07     zip_path = DIST_ROOT / f"{DIST_NAME}.zip"
08     if zip_path.exists():
09         zip_path.unlink()
10     tmp_zip_path = zip_path.with_suffix(zip_path.suffix + ".tmp")
11     if tmp_zip_path.exists():
12         tmp_zip_path.unlink()
13
14
15 def rename_pyinstaller_output() -> None:
16     generated = DIST_ROOT / APP_NAME
17     if not generated.exists():
18         raise FileNotFoundError(f"PyInstaller output not found: {generated}")
19     if DIST_DIR.exists():
20         shutil.rmtree(DIST_DIR)
21     generated.rename(DIST_DIR)
22
23
24 def make_zip() -> Path:
25     zip_path = DIST_ROOT / f"{DIST_NAME}.zip"
26     tmp_zip_path = zip_path.with_suffix(zip_path.suffix + ".tmp")
27     if zip_path.exists():
28         zip_path.unlink()
29     if tmp_zip_path.exists():
30         tmp_zip_path.unlink()
31
32     dirs = [DIST_DIR, *sorted(p for p in DIST_DIR.rglob("*") if p.is_dir())]
33     files = sorted(p for p in DIST_DIR.rglob("*") if p.is_file())
34     print(f"[info] Creating zip: {zip_path}", flush=True)
35     print(f"[info] Compressing {len(files)} files. This may take a while...", flush=True)
36
37     try:
38         with tmp_zip_path.open("wb") as raw:
39             with ZipFile(raw, "w", compression=ZIP_DEFLATED, compresslevel=6, allowZip64=True) as zf:
40                 for path in dirs:
41                     arcname = path.relative_to(DIST_ROOT).as_posix().rstrip("/")
42                     zf.write(path, arcname + "/")
43                 for index, path in enumerate(files, start=1):
44                     arcname = path.relative_to(DIST_ROOT).as_posix()
45                     zf.write(path, arcname)
46                     if index == len(files) or index % 250 == 0:
47                         print(f"[zip] {index}/{len(files)} files", flush=True)
48             raw.flush()
49             os.fsync(raw.fileno())
50

```

```

01     print("[info] Validating temporary zip...", flush=True)
02     validate_distribution(tmp_zip_path)
03     os.replace(tmp_zip_path, zip_path)
04     print("[info] Validating final zip...", flush=True)
05     validate_distribution(zip_path)
06 except Exception:
07     if tmp_zip_path.exists():
08         tmp_zip_path.unlink()
09     raise
10
11     return zip_path
12
13
14 def expected_zip_file_entries() -> set[str]:
15     return {
16         path.relative_to(DIST_ROOT).as_posix()
17         for path in DIST_DIR.rglob("*")
18         if path.is_file()
19     }
20
21
22 def expected_zip_dir_entries() -> set[str]:
23     return {
24         path.relative_to(DIST_ROOT).as_posix().rstrip("/") + "/"
25         for path in [DIST_DIR, *DIST_DIR.rglob("*")]
26         if path.is_dir()
27     }
28
29
30 def read_zip_entries(zip_path: Path) -> tuple[set[str], set[str]]:
31     with ZipFile(zip_path, "r") as zf:
32         bad_file = zf.testzip()
33         if bad_file:
34             raise ValueError(f"Zip entry failed CRC check: {bad_file}")
35         entries = set(zf.namelist())
36         file_entries = {entry for entry in entries if not entry.endswith("/")}
37         dir_entries = {entry for entry in entries if entry.endswith("/")}
38         return file_entries, dir_entries
39
40
41 def validate_distribution(zip_path: Path) -> None:
42     required_files = [
43         DIST_DIR / f"{APP_NAME}.exe",
44         DIST_DIR / APP_ICON,
45         DIST_DIR / "_internal" / APP_ICON,
46         DIST_DIR / "_internal" / "_tcl_data" / "init.tcl",
47         DIST_DIR / "_internal" / "_tk_data" / "tk.tcl",
48         DIST_DIR / "_internal" / "core" / "checkpoints" / "click1_siamese_yolo4_best.onnx",
49         DIST_DIR / "_internal" / "core" / "checkpoints" / "click3_yolo_plan_bg_char60_best.onnx",
50     ]

```

```

01     missing = [str(path) for path in required_files if not path.exists()]
02     if missing:
03         raise FileNotFoundError("Distribution is missing required files:\n" + "\n".join(missing))
04
05     required_entries = [
06         f"{DIST_NAME}/{APP_NAME}.exe",
07         f"{DIST_NAME}/{APP_ICON}",
08         f"{DIST_NAME}/_internal/{APP_ICON}",
09         f"{DIST_NAME}/_internal/_tcl_data/init.tcl",
10         f"{DIST_NAME}/_internal/_tk_data/tk.tcl",
11         f"{DIST_NAME}/_internal/core/checkpoints/click1_siamese_yolo4_best.onnx",
12         f"{DIST_NAME}/_internal/core/checkpoints/click3_yolo_plan_bg_char60_best.onnx",
13     ]
14     file_entries, dir_entries = read_zip_entries(zip_path)
15     entries = file_entries | dir_entries
16     missing_entries = [entry for entry in required_entries if entry not in entries]
17     if missing_entries:
18         raise FileNotFoundError("Zip is missing required entries:\n" + "\n".join(missing_entries))
19
20     expected_files = expected_zip_file_entries()
21     missing_files = sorted(expected_files - file_entries)
22     extra_files = sorted(file_entries - expected_files)
23     if missing_files:
24         raise FileNotFoundError("Zip is missing files from the dist folder:\n" + "\n".join(missing_files[:50]))
25     if extra_files:
26         raise ValueError("Zip contains unexpected files:\n" + "\n".join(extra_files[:50]))
27
28     expected_dirs = expected_zip_dir_entries()
29     missing_dirs = sorted(expected_dirs - dir_entries)
30     if missing_dirs:
31         raise FileNotFoundError("Zip is missing directories from the dist folder:\n" + "\n".join(missing_dirs[:50]))
32
33
34 def build(
35     index_url: str | None,
36     *,
37     app_name: str = APP_NAME,
38     app_version: str = APP_VERSION,
39     dist_name: str | None = None,
40 ) -> None:
41     configure_package_names(app_name, app_version, dist_name)
42
43     if sys.platform != "win32":
44         print("[warn] This script is intended for Windows exe packaging.")
45
46     py = build_python()
47     print(f"[info] Build Python: {py}")
48     print(f"[info] EXE name: {APP_NAME}.exe")
49     print(f"[info] Dist folder/zip name: {DIST_NAME}")
50     ensure_build_deps(py, index_url)

```

```

01     validate_icon_file()
02
03     clean_previous_output()
04     run(pyinstaller_cmd(py))
05     print("[info] PyInstaller finished. Preparing runtime files and zip package...", flush=True)
06     rename_pyinstaller_output()
07     write_runtime_files()
08     zip_path = make_zip()
09
10     print()
11     print("=" * 70)
12     print("Build complete")
13     print(f"Folder: {DIST_DIR}")
14     print(f"Zip:      {zip_path}")
15     print(f"Run:      {DIST_DIR / (APP_NAME + '.exe')}")
16     print("=" * 70)
17
18
19 def main() -> None:
20     parser = argparse.ArgumentParser(description="Build LNU-LibSeat Windows exe package.")
21     parser.add_argument(
22         "--index-url",
23         default=os.environ.get("PIP_INDEX_URL"),
24         help="Optional pip index URL, for example https://pypi.tuna.tsinghua.edu.cn/simple",
25     )
26     parser.add_argument(
27         "--app-name",
28         default=APP_NAME,
29         help=f"Executable name without .exe. Default: {APP_NAME}",
30     )
31     parser.add_argument(
32         "--app-version",
33         default=APP_VERSION,
34         help=f"Version used in the default dist name. Default: {APP_VERSION}",
35     )
36     parser.add_argument(
37         "--dist-name",
38         default=None,
39         help=f"Folder name under dist and zip filename without .zip. Default: <app-name>-<app-version>",
40     )
41     args = parser.parse_args()
42     build(
43         args.index_url,
44         app_name=args.app_name,
45         app_version=args.app_version,
46         dist_name=args.dist_name,
47     )
48
49
50 if __name__ == "__main__":

```

```

01     main()
02
03 # ===== FILE: build_mac.py =====
04 """
05 LNU_Assistant macOS 构建脚本
06 =====
07 把项目打包成可双击的 macOS ``.app``, 并装入一个发行夹后用 ``ditto`` 压成 zip。
08
09     PyInstaller 不能跨平台编译: 本脚本**必须在 macOS 上运行** (本地 Apple
10     Silicon Mac, 或 GitHub Actions 的 macos-l4 runner)。在 Windows 上无法产出 .app。
11
12 工作流程:
13     1. 创建干净的临时 venv (隔离 Anaconda / torch 等)
14     2. 只安装运行所需的包
15     3. 使用项目图标资源
16     4. 在干净 venv 内运行 PyInstaller, 产出 dist/LNU-LibSeat.app
17     5. 组装发行夹 (.app + 外置可编辑 config.py + info/ + logs/ + 解除限制脚本 + 说明)
18     6. 用 ditto 压成 zip (保留 .app 内的符号链接与签名)
19     7. 清理临时 venv
20
21 用法 (在 Mac 上):
22     python3 build_mac.py
23
24 产物:
25     dist/LNU-LibSeat-v2.6.0-macOS-arm64/      <- 发行夹
26     dist/LNU-LibSeat-v2.6.0-macOS-arm64.zip   <- 分发给用户的压缩包
27 """
28 import os
29 import platform
30 import shutil
31 import subprocess
32 import sys
33 import venv
34
35 ROOT = os.path.dirname(os.path.abspath(__file__))
36 APP_NAME = "LNU-LibSeat"
37 APP_ICON = "OIP-C.ico"
38 APP_VERSION = "v2.6.0" # 每次发布新版本请修改此处
39 ARCH = platform.machine() or "arm64" # arm64 (Apple Silicon)
40 DIST_NAME = f"{APP_NAME}-{APP_VERSION}-macOS-{ARCH}"
41 DIST_DIR = os.path.join(ROOT, "dist", DIST_NAME)
42 VENV_DIR = os.path.join(ROOT, ".build_venv_mac")
43 BUNDLE_ID = "com.xunrana.lnulibseat"
44
45 # 只把这些包 (及其依赖) 打进 .app
46 BUILD_DEPS = [
47     "pyinstaller", "PySide6", "selenium", "ddddocr",
48     "opencv-python", "numpy", "onnxruntime", "Pillow",
49     "requests", "webdriver-manager", "mss",
50 ]

```

```

01
02
03 def _venv_python():
04     return os.path.join(VENV_DIR, "bin", "python")
05
06
07 def _create_clean_venv():
08     if os.path.exists(VENV_DIR):
09         print("[*] 移除旧的 build venv...")
10         shutil.rmtree(VENV_DIR)
11
12     print("[*] 创建干净的 build venv...")
13     venv.create(VENV_DIR, with_pip=True)
14
15     py = _venv_python()
16     if not os.path.isfile(py):
17         sys.exit(f"[ERROR] venv python 未找到: {py}")
18
19     print("[*] 安装构建依赖 (arm64 wheel) ...")
20     subprocess.check_call([py, "-m", "pip", "install", "--upgrade", "pip"],
21                             stdout=subprocess.DEVNULL)
22     subprocess.check_call([py, "-m", "pip", "install"] + BUILD_DEPS)
23     print("[OK] build venv 就绪。\\n")
24
25
26 def _make_icns():
27     return None
28
29
30 # 干净的配置模板 (绝不打包个人数据; macOS 默认浏览器 = chrome)
31 CLEAN_CONFIG = '''\\
32 # =====
33 # LNU_Assistant 配置文件 (由 GUI 自动保存)
34 # =====
35
36 USERS = {
37     "": {
38         "password": "",
39         "time": {"start": "", "end": ""}
40     },
41 }
42
43 TARGET_CAMPUS = "崇山校区图书馆"
44 TARGET_ROOM = "三楼智慧研修空间"
45 PREFER_SEATS = []
46
47 WAIT_FOR_0630 = False
48 MAX_ACCOUNTS = 2
49
50 BROWSER = "chrome"

```

```

01 DRIVER_PATH = ""
02 WEBDRIVER_CACHE = ""
03
04 RECEIVER_EMAIL = ""
05 SMTP_USER = ""
06 SMTP_PASS = ""
07
08 LOG_LEVEL = "INFO"
09 LOG_DIR = "logs"
10 '''
11
12 # 一键解除 Gatekeeper 限制脚本（未签名 .app 首次需要）
13 UNLOCK_COMMAND = '''\
14 #!/bin/bash
15 # 解除 macOS 对未签名应用的隔离限制，然后启动应用。
16 cd "$(dirname "$0")"
17 echo "正在解除安全限制（可能需要几秒）..."
18 xattr -cr "LNU-LibSeat.app" 2>/dev/null
19 chmod +x "LNU-LibSeat.app/Contents/MacOS/LNU-LibSeat" 2>/dev/null
20 echo "完成！正在启动 LNU-LibSeat..."
21 open "LNU-LibSeat.app"
22 '''
23
24 USAGE_TXT = '''\
25 LNU-LibSeat 图书馆智能抢座系统 • macOS 使用说明
26 =====
27
28 【首次运行】
29 1. 把本文件夹整个解压到任意位置（例如「下载」或「桌面」）。
30 2. 右键点击「首次运行请先双击我.command」→ 选择「打开」→ 再点一次「打开」。
31    （因为应用未做苹果付费签名，必须用右键打开一次以解除安全限制；
32    之后即可直接双击 LNU-LibSeat.app 启动。）
33 3. 如果第 2 步提示「无法打开」，请到 系统设置 隐私与安全性 底部「仍要打开」。
34
35 【浏览器要求】
36 - 默认使用 Chrome，请先安装 Google Chrome（推荐，支持双账号并行）。
37 - 如果想用 Safari：在应用内「浏览器」选择 Safari，并先开启：
38   Safari 设置 高级 勾选「在菜单栏显示开发菜单」→ 开发 允许远程自动化。
39   （注意：Safari 只支持单账号，不支持双账号并行。）
40
41 【数据位置】
42 - config.py（你的配置）、logs/（运行日志）都在本文件夹内，与 .app 同级，可直接查看/编辑。
43
44 【注意】
45 - 抢座等待期间请保持电脑开机；应用会自动防止系统休眠。
46 - 屏幕录制功能在 macOS 上可能不可用（不影响抢座）。
47 '''
48
49
50 def build():

```



```

01     if sys.platform != "darwin":
02         sys.exit("[ERROR] build_mac.py 只能在 macOS 上运行。Windows 请用 build.py, "
03             "或通过 GitHub Actions 的 macos runner 构建。")
04
05     # --- Step 1: venv ---
06     _create_clean_venv()
07     py = _venv_python()
08
09     # --- Step 2: icns ---
10     icns = _make_icns()
11
12     # --- Step 3: PyInstaller ---
13     sep = os.pathsep # macOS = ":"
14     cmd = [
15         py, "-m", "PyInstaller",
16         "--noconfirm",
17         "--onedir",
18         "--windowed", # 生成 .app (GUI, 无终端窗口)
19         "--name", APP_NAME,
20         "--distpath", os.path.join(ROOT, "dist"),
21         "--osx-bundle-identifier", BUNDLE_ID,
22
23         # 运行时 hook: 在任何 import 前把 cwd / sys.path 指向 .app 同级目录
24         "--runtime-hook", os.path.join(ROOT, "_runtime_hook.py"),
25
26         # 随包资源
27         "--add-data", f"{os.path.join(ROOT, APP_ICON)}{sep}. ",
28         # 验证码 ONNX 模型 -> _MEIPASS/core/checkpoints (由 core.paths.resource_root 定位)
29         "--add-data", f"{os.path.join(ROOT, 'core', 'checkpoints')}{sep}core/checkpoints",
30
31         # 带原生库 / 数据文件的包
32         "--collect-all", "ddddocr",
33         "--collect-all", "onnxruntime",
34         "--collect-all", "selenium",
35         "--collect-all", "PySide6",
36         "--collect-all", "cv2",
37         "--collect-all", "numpy",
38
39         # 不打包 config.py (用户编辑外置副本)
40         "--exclude-module", "config",
41         # 排除运行时用不到的训练 / 推理重型依赖
42         "--exclude-module", "torch",
43         "--exclude-module", "torchvision",
44         "--exclude-module", "timm",
45         "--exclude-module", "scipy",
46         "--exclude-module", "matplotlib",
47         "--exclude-module", "pandas",
48         "--exclude-module", "ultralitics",
49         "--exclude-module", "tensorboard",
50         "--exclude-module", "tensorflow",

```

```

01 ]
02 if icns:
03     cmd += ["--icon", icns]
04 cmd.append(os.path.join(ROOT, "gui_qt.py")) # 入口
05
06 print("=" * 55)
07 print(" Building LNU-LibSeat (macOS .app) ...")
08 print("=" * 55)
09 subprocess.check_call(cmd)
10
11 # --- Step 4: 清理 venv ---
12 print("[*] 清理 build venv...")
13 shutil.rmtree(VENV_DIR, ignore_errors=True)
14
15 app_src = os.path.join(ROOT, "dist", f"{APP_NAME}.app")
16 if not os.path.isdir(app_src):
17     sys.exit(f"[ERROR] 未生成 .app: {app_src}")
18
19 # --- Step 5: 组装发行夹 ---
20 if os.path.exists(DIST_DIR):
21     shutil.rmtree(DIST_DIR)
22 os.makedirs(DIST_DIR)
23
24 # .app 移入发行夹
25 shutil.move(app_src, os.path.join(DIST_DIR, f"{APP_NAME}.app"))
26 print(f"[OK] {APP_NAME}.app 已放入发行夹")
27 # PyInstaller 同时产生的 onedir 文件夹（非 .app）删掉，避免混淆
28 shutil.rmtree(os.path.join(ROOT, "dist", APP_NAME), ignore_errors=True)
29
30 # 外置可编辑 config.py（与 .app 同级 -> 对应 core.paths.app_data_dir）
31 with open(os.path.join(DIST_DIR, "config.py"), "w", encoding="utf-8") as f:
32     f.write(CLEAN_CONFIG)
33 print("[OK] 干净 config.py 模板已写入（无个人数据）")
34
35 # info/（座位索引）
36 src_info = os.path.join(ROOT, "info")
37 if os.path.exists(src_info):
38     shutil.copytree(src_info, os.path.join(DIST_DIR, "info"))
39     print("[OK] info/ 已复制")
40
41 # logs/
42 os.makedirs(os.path.join(DIST_DIR, "logs"), exist_ok=True)
43 print("[OK] logs/ 已创建")
44
45 # 解除限制脚本
46 cmd_path = os.path.join(DIST_DIR, "首次运行请先双击我.command")
47 with open(cmd_path, "w", encoding="utf-8") as f:
48     f.write(UNLOCK_COMMAND)
49 os.chmod(cmd_path, 0o755)
50 print("[OK] 解除限制脚本已写入")

```

```

01
02  # 使用说明
03  with open(os.path.join(DIST_DIR, "使用说明.txt"), "w", encoding="utf-8") as f:
04      f.write(USAGE_TXT)
05  print("[OK] 使用说明.txt 已写入")
06
07  # --- Step 6: ditto 压 zip (保留 .app 符号链接 / 权限 / ad-hoc 签名) ---
08  zip_path = os.path.join(ROOT, "dist", f"{DIST_NAME}.zip")
09  if os.path.exists(zip_path):
10      os.remove(zip_path)
11  print(f"[*] 用 ditto 打包 {DIST_NAME}.zip ...")
12  subprocess.check_call([
13      "ditto", "-c", "-k", "--sequesterRsrc", "--keepParent",
14      DIST_DIR, zip_path,
15  ])
16  print(f"[OK] {DIST_NAME}.zip 创建成功")
17
18  print()
19  print("=" * 55)
20  print("  构建完成!")
21  print(f"  发行夹: dist/{DIST_NAME}/")
22  print(f"  压缩包: dist/{DIST_NAME}.zip")
23  print()
24  print("  分发给用户: 发送 zip; 用户解压后右键运行")
25  print("  「首次运行请先双击我.command」即可。")
26  print("=" * 55)
27
28
29  if __name__ == "__main__":
30      build()
31
32  # ===== FILE: build_exe.bat =====
33  @echo off
34  chcp 65001 >nul
35  setlocal
36  cd /d "%~dp0"
37
38  if exist "env\Scripts\python.exe" (
39      set "PY=env\Scripts\python.exe"
40  ) else (
41      set "PY=python"
42  )
43
44  echo =====
45  echo   Building LNU-LibSeat exe package
46  echo =====
47  "%PY%" build.py --index-url https://pypi.tuna.tsinghua.edu.cn/simple %*
48
49  echo.
50  if %errorlevel% neq 0 (

```

```

01     echo [FAIL] Build failed.
02 ) else (
03     echo [OK] Build finished. See dist folder.
04 )
05 pause
06
07 # ===== FILE: run.bat =====
08 @echo off
09 chcp 65001 >nul
10 setlocal
11 cd /d "%~dp0"
12
13 :: Check Python
14 python --version >nul 2>&1
15 if %errorlevel% neq 0 (
16     echo =====
17     echo [ERROR] Python not found!
18     echo Please install Python 3.8+ from https://www.python.org/
19     echo Make sure to check "Add python.exe to PATH"
20     echo =====
21     pause
22     exit /b
23 )
24
25 :: Setup venv if not exists
26 if not exist "env\Scripts\python.exe" (
27     echo =====
28     echo [INFO] First run - setting up virtual environment...
29     echo =====
30     python -m venv env
31     echo.
32     echo [INFO] Installing dependencies...
33     env\Scripts\pip.exe install selenium webdriver-manager requests Pillow ddddocr PySide6 mss opencv-python onnxruntime -i https://pypi. ...
34     echo.
35     echo =====
36     echo [OK] Environment ready!
37     echo =====
38     timeout /t 2 >nul
39 )
40
41 echo.
42 echo =====
43 echo   LibSeat Allocator Starting...
44 echo =====
45 start "LibSeat Allocator" env\Scripts\pythonw.exe app.py
46
47 echo.
48 echo App launched! You can close this window.
49 timeout /t 3 >nul
50

```